



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione
Corso di Laurea Magistrale in Ingegneria Robotica e dell'Automazione

Elaborato finale del corso **Non-Linear Dynamics and Control**

***Analysis and Control of a Non-Linear
Active Suspension System***

Anno Accademico 2025/2026

Candidato
Arsen Hudyma
matr. P38000269

Index

Introduction	1
1 Context	2
1.1 Quarter-Car Model	2
1.2 Modeled Scenario	3
1.3 Mathematical Model	4
1.3.1 Non-Linear Spring	4
1.4 Parameters	7
1.5 Input-State-Output Form	8
2 Open-Loop Analysis	12
2.1 Study of Equilibrium	12
2.2 Lyapunov's Direct Method, Enforced with La Salle	16
2.3 Passive Suspension's Response to Road Profile	18
2.3.1 Road Bump	20
2.3.2 Off-Road	21
3 Synthesis	24
3.1 Control Formulation	24
3.2 LQR	25
3.2.1 Road Bump	28
3.2.2 Off-Road	29
3.3 NL MPC	32
3.3.1 Road Bump	35

3.3.2	Off-Road	36
3.4	Extra, Linear Spring	38
4	Robustness Analysis	42
4.1	Parametric Uncertainties	42
4.1.1	Road Bump	42
4.1.2	Off-Road	45
4.2	Disturbance	50
4.2.1	Road Bump	51
4.2.2	Off-Road	53
4.2.3	Extra, Saturation	57
5	Conclusion	61
5.1	Passive vs Active	61
5.2	Degradation of Performance	62
5.3	Final Discussion	63
	Bibliography	64

Abstract

This project was born by means of two needs: of course the need to study and control a Non-Linear system via some of the techniques that we studied during the course, but also the other (personal) need of applying the seen theory to something that I already really enjoyed, such as to obtain a combination of passion and interest to do (hopefully) a good job. The field of interest will be the automotive industry.

In the *first* chapter I will illustrate the modeled scenario and its mathematical description.

Then, in the *second* chapter we will perform an open-loop analysis on the system.

The *third* chapter will focus on the synthesis of control schemes for the system, in particular on a Linear and a Non-Linear ones.

In the *fourth* chapter we will instead testing the robustness of the implemented controllers.

In the *fifth* and last chapter we'll do a comparison between the schemes and will make conclusions.

Chapter 1

Context

The goal of this first chapter will be to make you familiar with the chosen plant for this project, its peculiarities and different ways in which it can be used. Of course we'll focus on the mathematical model and in fact there will be a discussion regarding the meaning and need of the various terms and parameters with natural extension from physics to mathematics, but also you'll be aware of why the need of Non-Linear terms in it. The last point to address will be the formulation of the ISO (Input-State-Output) form that we'll need for analysis and synthesis in the next chapters.

1.1 Quarter-Car Model

This model is one of the simplest that is been used in the automotive industry to study and simulate the suspension system of a car, the name speaks by itself, in fact we'll focus only on a single car's tire and its relative suspension, like we can see:

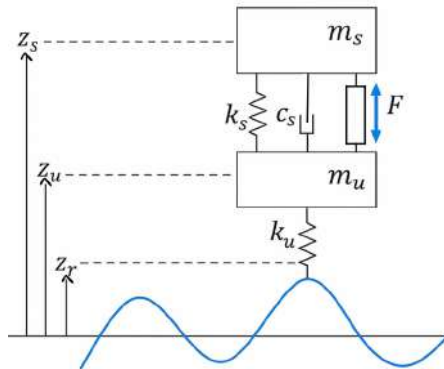


Figure 1.1: Quarter-Car Scheme

in fact it's a 4 DoF system consisting in two masses connected by means of springs and dampers. The upper mass, also known in the literature as *sprung mass* m_s represents the car (quart of its total mass), the lower one which is called *unsprung mass* m_u represent the tire plus its suspension and we consider their vertical excursion by means of z_s and z_u which are centered to the relative centroids. The road surface is described via z_r . Then the tire's stiffness is modeled by k_u while the suspension's spring stiffness is represented via k_s and a damper c_s . The last component in the scheme, represented by F is actually the active element (typically an hydraulic piston):



Figure 1.2: Active Pistons

We can notice that only the vertical dynamics will be highlighted, other phenomena born by the mutual interactions between the wheels and the road like roll, pitch and yaw will not be present due to the fact that we're not considering the full-car or half-car model. The questions to address now are: "What scenario we can implement with this plant?" and "This mathematical description will be enough?".

1.2 Modeled Scenario

We could simply perform a height control on the car, which can be used in racing scenario when approaching curves as in F1 or Hypercar racing, where the height is important to control the airflow under the car to optimize performance and stability (maybe combined with other wheels) or (which is the actual case) we can aim to improve the comfort of the passengers by trying to limit as much as we can the vertical excursion and acceleration of the car while keeping the contact between the tire and the road for stability. In this case the quarter car model will be sufficient since the modeled scenario will consist of the car moving forward at low velocity on an irregular road surface[1].

1.3 Mathematical Model

Now we just need to write the dynamics for our system, if we indicate with L_s and L_u the unstretched lengths of the springs, we obtain:

$$m_s \ddot{z}_s(t) = k_s \Delta_{z1} (1 + \epsilon_s \Delta_{z1}^2) + c_s \dot{\Delta}_{z1} + k_s L_s - m_s g + F(t), \quad (1.1)$$

$$\begin{aligned} m_u \ddot{z}_u(t) = & -k_s \Delta_{z1} (1 + \epsilon_s \Delta_{z1}^2) - c_s \dot{\Delta}_{z1} - k_s L_s \\ & + k_u \Delta_{z2} + k_u L_u - m_u g - F(t) \end{aligned} \quad (1.2)$$

$$\Delta_{z1} = z_u(t) - z_s(t), \quad (1.3)$$

$$\dot{\Delta}_{z1} = \dot{z}_u(t) - \dot{z}_s(t), \quad (1.4)$$

$$\Delta_{z2} = z_r(t) - z_u(t). \quad (1.5)$$

Considering for simplicity the first equation, we can notice that it's very similar to a classic Mass-Spring-Damper system formulation, expect for the $k_s L_s$ term and the parenthesis. The first one is needed to preserve respectively the positions of the centroids of m_s and m_u , otherwise they would be referred to zero.

Then the most interesting part, the Non-Linear term that models the stiffness of the spring as a linear term plus a cubic one, in fact $k_s \epsilon_s$ can be seen as a non-linear stiffness coefficient k_{nl} :

$$k_s \Delta_{z1} (1 + \epsilon_s \Delta_{z1}^2) = k_s \Delta_{z1} + k_s \epsilon_s \Delta_{z1}^3 = k_s \Delta_{z1} + k_{nl} \Delta_{z1}^3 \quad (1.6)$$

1.3.1 Non-Linear Spring

The stiffness formulation seen is relative to the Duffing's oscillator[2], by which we are enhancing the Quarter-Car model. In function of the sign of k_{nl} we are modeling the hardening/softening effect of a spring, which consist in having a bigger/lower response force than the expected one after a certain threshold:

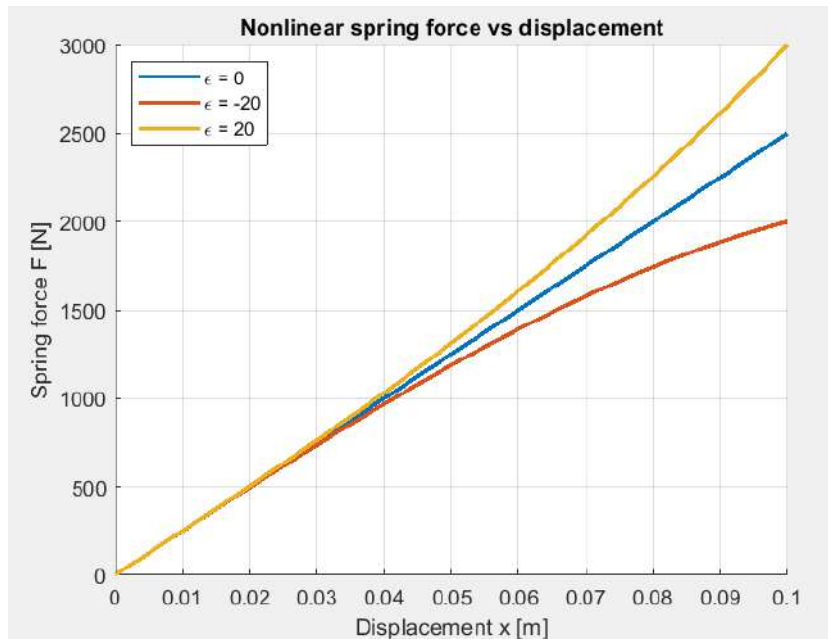


Figure 1.3: Linear Spring (blue), Hardening (yellow) and Softening (orange)

The question to address now is: "Does it make sense to model the spring in this way?"

In my researches in the literature I've seen that effectively the linear model of the spring actually is a simplification, there are many different and more realistic ways to model a spring. Someone did some experimental test on a spring and showed that a quadratic term is necessary to correctly fit the obtained data[3]:

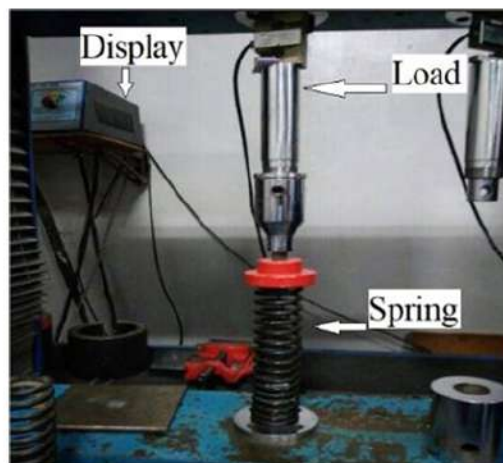


Figure 1.4: Experiment Setup

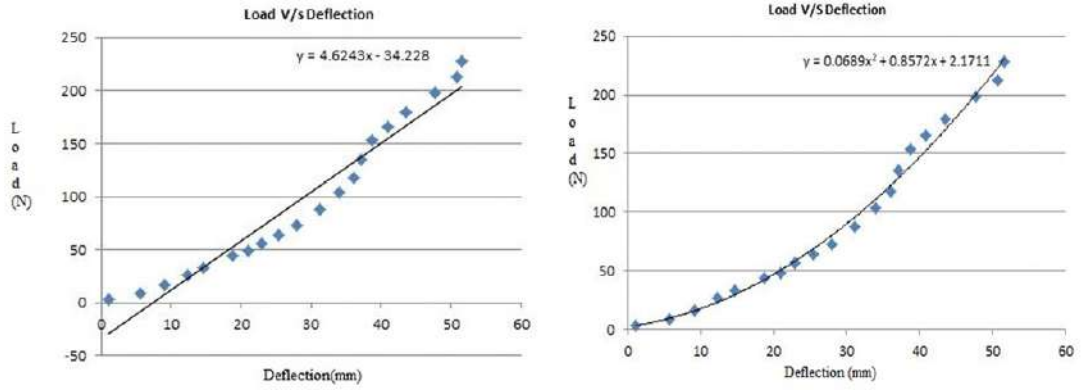


Figure 1.5: Displacement/Force: Linear (L) and Non-Linear (R)

Other studies highlighted that it would be reasonable to model the spring with hysteresis[4]:

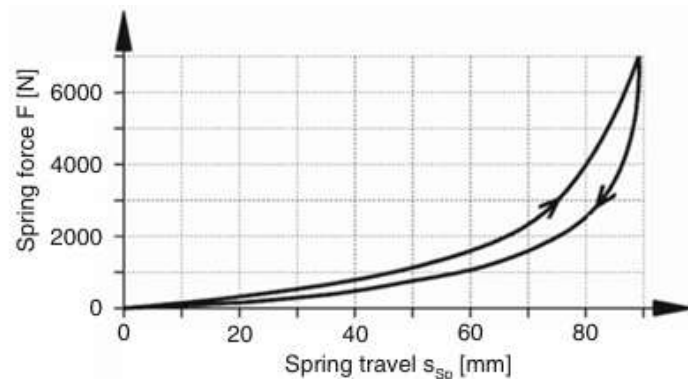


Figure 1.6: Hysteretical Behavior of a Spring

Instead, sometimes it is a desired behavior obtained by construction, it's the case of *progressive* springs¹. Generally, the distance between each coil of a spring is constant throughout its length, linear springs are made in this way. Instead by changing the distance between the coils one can obtain a spring that becomes stiffer by increasing its deformation. This can be an advantage because we can obtain a spring by which we realize a trade-off between a smooth shock absorption of an imperfect road and a good road handling for stability of the vehicle.

¹You can search for the *Eibach* catalog or the Monroe's one



Figure 1.7: Linear VS Progressive springs

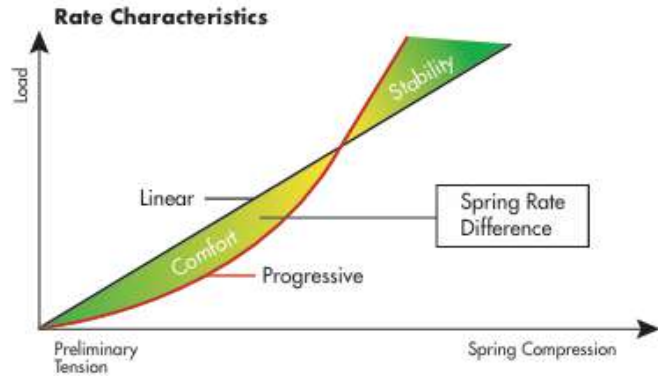


Figure 1.8: Force-Displacement comparison

In this project I've decided to model a springs with hardening effect via the Duffing's model (this means $k_{nl} > 0$), due to the lack of F/d diagrams because they're private for each company I'll chose the ϵ_s in a way that the Non-Linear effects will be highlighted in the example. The difference from Linear and NonLinear behavior of the spring will be:

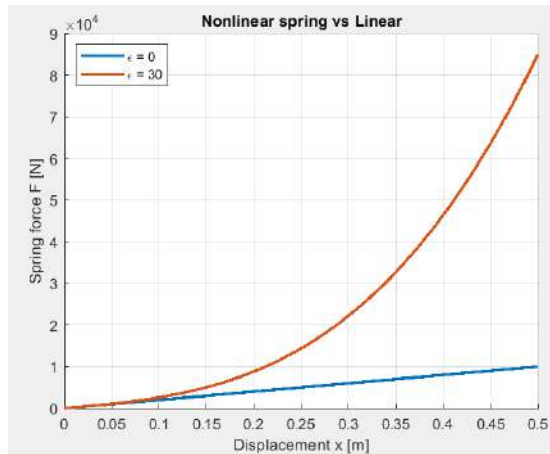


Figure 1.9: L vs NL Spring

1.4 Parameters

Since there's the need to chose the vehicle's parameters, I've decided to take as example the Mercedes GLC Coupè AMG Performance:

- $m_s = 2310/4 \approx 577$ kg
- $m_u = 50$ kg
- $k_s = 2 \cdot 10^4$ N/m
- $k_u = 2.5 \cdot 10^5$ N/m
- $c_s = 2 \cdot 10^3$ N/(m/s)
- $L_s = 0.40$ m
- $L_u = 0.34$ m
- $\epsilon_s = 30$ m⁻²



Figure 1.10: GLC Coupé AMG



Figure 1.11: Suspension System



Figure 1.12: Wheel

1.5 Input-State-Output Form

Now the last thing to address regards reformulating the system's equation in *affine form* so we'll need to highlight the states $\mathbf{x}$, the inputs $\mathbf{u}$ and the outputs $\mathbf{y}$ of the system, and of course also their relative vector fields. The general affine form is:

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x}) \mathbf{u} \quad (1.7)$$

$$\mathbf{y} = h(\mathbf{x}) \quad (1.8)$$

we've already seen that the system has 4 DoF, this means that we'll need to know the positions and velocities of the two masses, so the state vector will be:

$$\mathbf{x} = \begin{bmatrix} z_s \\ z_u \\ \dot{z}_s \\ \dot{z}_u \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

and as input variables we will have respectively the actuation force $F(t)$ which will compensate for the road profile $z_r(t)$ which is actually the second input of the system. In fact in this project we'll assume to know the road profile, which is of course a strong assumption, but is motivated by:

- if you experienced a drive in a modern car (such as a Mercedes), it could happen that the virtual assistant warns you about a pothole. This means that the car is provided with some sensors that can recognize them. In detail, Mercedes-Benz have patented the "Magic Body Control" (MBC)² which is a system of a stereo camera by which the car can recognize a road irregularity in real time with a precision of about 3 *mm*:



Figure 1.13: Camera in the MBC

²<https://youtu.be/ScpgI1w5F6A?si=cVHBLhYRpDTo4ITY>

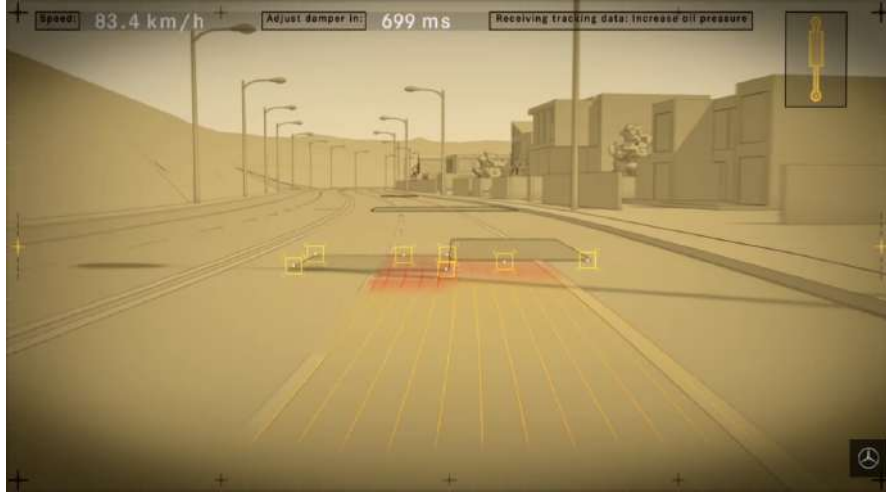


Figure 1.14: Estimation of the MBC



Figure 1.15: MBC at work

After all of this motivation, we can say that our inputs are:

$$\mathbf{u} = \begin{bmatrix} F(t) \\ z_r(t) \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

then, based on the equations (1.1)-(1.5) we rewrite the system in the form:

$$\dot{x}_1 = x_3 \quad (1.9)$$

$$\dot{x}_2 = x_4 \quad (1.10)$$

$$\dot{x}_3 = \left(K_s(x_2 - x_1)(1 + \epsilon_s(x_2 - x_1)^2) + c_s(x_4 - x_3) + K_s L_s - m_s g + u_1 \right) m_s^{-1} \quad (1.11)$$

$$\begin{aligned} \dot{x}_4 = & \left(-K_s(x_2 - x_1)(1 + \epsilon_s(x_2 - x_1)^2) - c_s(x_4 - x_3) - K_s L_s - m_u g - u_1 \right. \\ & \left. + K_u(u_2 - x_2) + K_u L_u \right) m_u^{-1} \end{aligned} \quad (1.12)$$

Thus:

$$f(\mathbf{x}) = \begin{bmatrix} x_3 \\ x_4 \\ \left(K_s(x_2 - x_1)(1 + \epsilon_s(x_2 - x_1)^2) + c_s(x_4 - x_3) + K_s L_s - m_s g \right) m_s^{-1} \\ \left(-K_s(x_2 - x_1)(1 + \epsilon_s(x_2 - x_1)^2) - c_s(x_4 - x_3) - K_s L_s - m_u g - K_u x_2 + K_u L_u \right) m_u^{-1} \end{bmatrix} \quad (1.13)$$

$$g(\mathbf{x}) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ m_s^{-1} & 0 \\ -m_s^{-1} & K_u/m_u \end{bmatrix} \quad (1.14)$$

The vector of the states gives us all the needed information, especially the two positions of the masses, to verify that the excursion of the sprung mass (car) is moderate to guarantee comfort, meanwhile that the unsprung one (wheel) follows the road profile, due to stability issues. This means that the specification of $h(\mathbf{x})$ is not necessary, at least now.

Chapter 2

Open-Loop Analysis

Now that the system of interest has been introduced we can see the application of some of the concepts of the course to the open-loop analysis of this system. Obviously the first thing to address regards the study of the equilibrium points of the system, then by means of linearized system we will be able to classify them. Another relevant aspect will be proving GAS property for this system, which is a very logical think but harder to prove as you'll see. Then we'll set up our reference performance when the suspension system is passive, so without compensating for anything, in this way we will be able to do a comparison between the passive system versus the active system with the implementation of the controllers in chapter 3.

2.1 Study of Equilibrium

The first thing that we've learned during the course was about the possibility that our system could have multiple equilibrium points with respect to the linear case. As well known, all we have to do is set our differential equations to zero, because logically at an equilibrium point the system will be in static conditions, and so the derivatives of the state variables will be null. And of course in the open-loop analysis we will consider our input variables $\mathbf{u} = 0$, thus:

$$\dot{x}_1 = f_1(\mathbf{x}) = 0 = x_3 \quad (2.1)$$

$$\dot{x}_2 = f_2(\mathbf{x}) = 0 = x_4 \quad (2.2)$$

$$\dot{x}_3 = f_3(\mathbf{x}) = 0 = k_s \Delta_{z1}(1 + \epsilon_s \Delta_{z1}^2) + k_s L_s - m_s g \quad (2.3)$$

$$\dot{x}_4 = f_4(\mathbf{x}) = 0 = -k_s \Delta_{z1}(1 + \epsilon_s \Delta_{z1}^2) - k_s L_s - k_u z_u + k_u L_u - m_u g \quad (2.4)$$

Thought the help of MatLab and in particular the symbolic toolbox it's quite easy to determine what are the solutions in term of $[x_1, x_2, x_3, x_4]^T$:

```
syms x1 x2 x3 x4 real
f1 = x3;
f2 = x4;
f3 = (ks*(x2-x1)*(1+epsilon*(x2-x1)^2)+cs*(x4-x3)+ks*ls-ms*g)/ms;
f4 = (-ks*(x2-x1)*(1+epsilon*(x2-x1)^2)-cs*(x4-x3)-ks*ls-mu*g-ku*x2+ku*lu).
    /mu;

sol = vpasolve([f1 == 0, f2 == 0, f3 == 0, f4 == 0], [x1, x2, x3, x4]);

disp('Equilibrium points:');

T = struct2table(sol);
T = round(T,2);    % 2 decimal digits
disp(T);
```

Figure 2.1: equilibria.m script

```
Equilibrium points:
      x1      x2      x3      x4
      ———      ———      —      —
      0.41      0.32      0      0
```

Figure 2.2: Solution

So, sadly there's only one solution and maybe it's not really interesting from the analysis viewpoint but this is the physics in this case. Because of course *the* equilibrium point of a quarter car system which models only the vertical dynamics, when there are not applied inputs to the system, represents the only one static condition in which on the suspension system and on the wheel acts the gravitational force. So we can already think that the equilibrium is at least locally stable but let's check it. To do so we have to define the *Jacobian*:

$$J(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \frac{\partial f_1}{\partial x_4} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \frac{\partial f_2}{\partial x_4} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} & \frac{\partial f_3}{\partial x_4} \\ \frac{\partial f_4}{\partial x_1} & \frac{\partial f_4}{\partial x_2} & \frac{\partial f_4}{\partial x_3} & \frac{\partial f_4}{\partial x_4} \end{bmatrix} \quad (2.5)$$

And in particular:

$$J(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{k_s}{m_s}(1 + \epsilon_s 3(x_2 - x_1)^2) & \frac{k_s}{m_s}(1 + \epsilon_s 3(x_2 - x_1)^2) & -\frac{c_s}{m_s} & \frac{c_s}{m_s} \\ \frac{k_s}{m_u}(1 + \epsilon_s 3(x_2 - x_1)^2) & -\frac{k_s}{m_u}(1 + \epsilon_s 3(x_2 - x_1)^2) - \frac{k_u}{m_u} & \frac{c_s}{m_u} & -\frac{c_s}{m_u} \end{bmatrix} \quad (2.6)$$

Now, all we have to do is evaluate the Jacobian at the equilibrium:

$$J(\mathbf{x}_A) = \left. \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_A}, \quad \mathbf{x}_A = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0.41 \\ 0.32 \\ 0 \\ 0 \end{bmatrix} \quad (2.7)$$

And thanks to MatLab:

```
J (xA) :
      0      0      1      0
      0      0      0      1
 -74.913   74.913  -3.4662   3.4662
  864.5  -5864.5    40    -40
```

Figure 2.3: Jacobian Evaluated at x_A

Now instead we can see the real information that we were looking for:

```
Eigenvalues of J (xA) :
 -20.462 +   73.156i
 -20.462 -   73.156i
  -1.2715 +    7.9558i
  -1.2715 -    7.9558i
```

Figure 2.4: Eigenvalues

As expected, the equilibrium is locally stable and in particular it's a *spiral sink*. Thanks also to intuition we can say that it's logic, because the two masses will oscillate until the total energy will be dissipated through the damping term and the system will stabilize at the equilibrium x_A .

Of course it is also an *isolated* equilibrium and *hyperbolic*. This means that we can apply the Hartman-Grossmann Theorem, so now let's see an open-loop

simulation with initial conditions that are disposed randomly far from the equilibrium point, with the evolution of the states and the phase portraits of x_1, x_3 and x_2, x_4 :

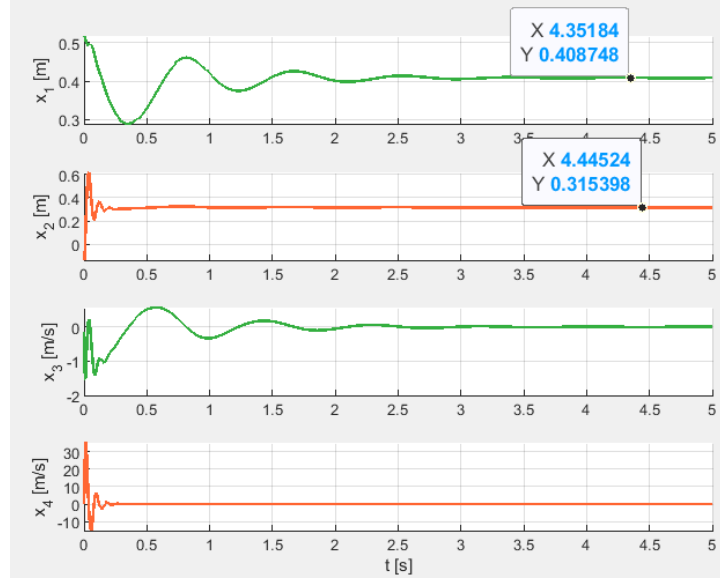


Figure 2.5: Evolution of the states with perturbation from x_A

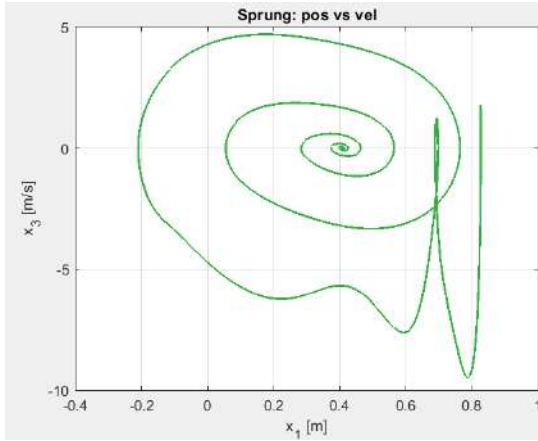


Figure 2.6: Phase Portrait of the sprung mass

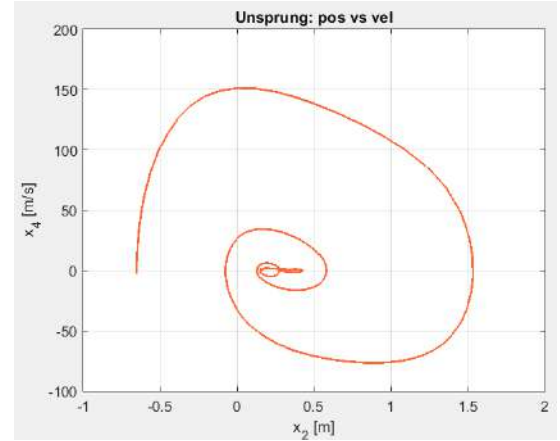


Figure 2.7: Phase Portrait of the unsprung mass

We can say that we're seeing what we actually imagined, because it must be like this, so the masses will settle down to the respective equilibrium points after some oscillations. Something more interesting regards the behavior of each mass, we've seen that there's a dominant eigenvalue with a lower imaginary part, and from picture 2.5 we can see that it corresponds to the sprung mass, on the other hand the faster couple of complex eigenvalues is related to the unsprung mass.

In fact by construction, we desire that the car moves smoothly, while the wheels need to respond faster to be in contact with the road.

2.2 Lyapunov's Direct Method, Enforced with La Salle

Until now we've established that the equilibrium is a stable focus, and by intuition we could say that we expect the system to be globally asymptotically stable, because when we're not providing energy to it it will for sure dissipate energy, but we need to prove it.

Here comes Lyapunov's stability and in particular the direct method, so we'll need a Lyapunov's function $V(\mathbf{x})$ that needs to satisfy:

- $V(\mathbf{x})$ has to be a C^1 function;
- $V(\mathbf{x}_A) = 0$;
- $V(\mathbf{x}) > 0 \quad \forall \mathbf{x} \in \mathbf{R}^n, \mathbf{x} \neq \mathbf{x}_A$;
- $\dot{V}(\mathbf{x}) \leq 0 \quad \forall \mathbf{x} \in \mathbf{R}^n, \mathbf{x} \neq \mathbf{x}_A$;

And a natural candidate function in this case is the total mechanical energy of the system, but traslated to the equilibrium, since in this case it is not the origin:

$$\tilde{\mathbf{x}} = \begin{bmatrix} x_1 - x_{1,A} \\ x_2 - x_{2,A} \\ x_3 - x_{3,A} \\ x_4 - x_{4,A} \end{bmatrix} = \begin{bmatrix} x_1 - x_{1,A} \\ x_2 - x_{2,A} \\ x_3 \\ x_4 \end{bmatrix}, \quad x_3 = \tilde{x}_3, \quad x_4 = \tilde{x}_4 \quad (2.8)$$

$$\begin{aligned} V(\tilde{x}) = & \frac{1}{2}m_s x_3^2 + \frac{1}{2}m_u x_4^2 \\ & + \frac{1}{2}k_s(\tilde{x}_2 - \tilde{x}_1)^2 + \frac{1}{4}k_s\epsilon(\tilde{x}_2 - \tilde{x}_1)^4 \\ & + \frac{1}{2}k_u \tilde{x}_2^2 \end{aligned} \quad (2.9)$$

We can notice that this V is also radially unbounded:

- $V(\mathbf{x}) \rightarrow \infty \quad \text{as} \quad \|\mathbf{x}\| \rightarrow \infty$

Now we have to focus instead on the derivative of V :

$$\dot{V}(\tilde{\mathbf{x}}) = \nabla V(\tilde{\mathbf{x}})f(\tilde{\mathbf{x}}) \quad (2.10)$$

which I've computed symbolically through MatLab:

```
% 2. Lyapunov function with traslated coordinates
delta = x2_tilde - x1_tilde;
V = 1/2*ms*x3_tilde^2 + 1/2*mu*x4_tilde^2 ...
    + 1/2*ks*delta^2 + 1/4*ks*epsilon*delta^4 ...
    + 1/2*ku*x2_tilde^2;

% 3. System in traslated coordinates
f1 = x3_tilde;
f2 = x4_tilde;
f3 = (ks*(x2_tilde - x1_tilde)*(1 + epsilon*(x2_tilde - x1_tilde)^2) ...
    + cs*(x4_tilde - x3_tilde))/ms;
f4 = (-ks*(x2_tilde - x1_tilde)*(1 + epsilon*(x2_tilde - x1_tilde)^2) ...
    - cs*(x4_tilde - x3_tilde) - ku*x2_tilde)/mu;

f = [f1; f2; f3; f4];
X_tilde = [x1_tilde; x2_tilde; x3_tilde; x4_tilde];

% 4. V dot symbolically computed
Vdot = simplify(jacobian(V,X_tilde)*f)
```

Figure 2.8: Lyap.m based on the traslated system

thus the result is:

$$\begin{aligned} \text{Vdot} = \\ -cs*(x3_tilde - x4_tilde)^2 \end{aligned}$$

Figure 2.9: $\dot{V}$ computed through Lyap.m

we can say for sure that $V(\tilde{\mathbf{x}})$ is not increasing, thus the energy of the system. But since the inequality $\dot{V}(\tilde{\mathbf{x}}) \leq 0$ does not hold strictly, thus the derivative is negative semi-definite we can only state that the system is locally stable, for sure not asymptotically.

But in our help comes the La Salle's invariance principle. In this case the set E is of course:

$$E = \{\tilde{\mathbf{x}} \in \mathbb{R}^4 : \dot{V}(\tilde{\mathbf{x}}) = 0\} = \{\tilde{\mathbf{x}} \in \mathbb{R}^4 : x_4 = x_3\} \quad (2.11)$$

The compact invariant set can be defined as:

$$\Omega_l := \{\tilde{\mathbf{x}} \in \mathbb{R}^n \mid V(\tilde{\mathbf{x}}) \leq l\}, \quad \text{for any } l > 0. \quad (2.12)$$

Then, by La Salle's theorem, all of the trajectories starting in Ω_l will converge to the largest invariant subset $\mathcal{M}$ within E . Thus, to identify $\mathcal{M}$ we'll analyze the trajectories starting from E , as always through MatLab's symbolic toolbox:

```

% 2. Translated system (deviation coordinates)
f3 = (ks*(x2 - x1)*(1 + epsilon*(x2 - x1)^2) + cs*(x4 - x3))/ms;
f4 = (-ks*(x2 - x1)*(1 + epsilon*(x2 - x1)^2) - cs*(x4 - x3) - ku*x2)/mu;

% 3. LaSalle condition: x4 = x3
x4_eq = x3;

% 4. Derivative along E: d(x4-x3)/dt = 0
f_diff = subs(f4 - f3, x4, x4_eq);
eqns = f_diff == 0;

% 5. Solve symbolically for x1, x2 (x3 remains symbolic)
vars = [x1, x2];
sol = solve(eqns, vars, 'Real', true);

```

Figure 2.10: La Salle.m based on the traslated system

```

Symbolic solutions inside E:
x1 = 0, x2 = 0, x3 = x3, x4 = x3

```

Figure 2.11: La Salle's solution

But if $\tilde{x}_1 = 0$ and $\tilde{x}_2 = 0$ then their derivatives must also be zero, so $\tilde{x}_3 = 0$ and $\tilde{x}_4 = 0$. Lastly, if we remember that we're working on the traslated system then of course the solution must be the equilibrium point itself: x_A which is also Globally Asymptotically Stable[5].

Before starting the next section, you may wonder "Why in the energy like Lyapunov's equation we have not considered gravity?" I think that the answer regards the fact that for the entire section we were working on the traslated system, so we could let's say "forget" gravity. Also because by intuition it clear that the system is GAS, but it was a little harder to prove since there were also other terms like $L_s K_s$ for example, which ones were needed to model the fact that the system would not have equilibrium ad $\mathbf{x}_A = 0$.

2.3 Passive Suspension's Response to Road Profile

This last section will be extremely useful because it will shows us the basic response of our system in the case when we're not generating a counter-balancing force $F(t)$. In detail we'll see the two reference road profiles $z_r(t)$ that will be used for the entire project and we'll focus on the response in terms of x_1 and x_2 . Graphics are good, but we'll need some other performance indexes, and in particular:

- regarding the comfort, if we consider that for $z_r = 0$, $\bar{z}_s = 0.41m$, then our interest is that independently from the road conditions, we would like that the distance between the actual value z_s and the equilibrium one $\bar{z}_s = 0.41$

to be the lowest as possible. We'll measure this information with the RMS operator:

$$\text{RMS}_{z_s} = \sqrt{\frac{1}{T} \int_0^T (z_s(t) - \bar{z}_s)^2 dt}$$

- instead regarding the safety, we would like to have the distance between the wheel z_u and the road z_r as lower as possible, and so being at least $z_u \approx z_r + \bar{z}_u = z_r + 0.32$ which will guarantee that the tire will be in contact with the road (considering the offset, because we're considering the barycenter of the wheel), so:

$$\text{RMS}_{u,r} = \sqrt{\frac{1}{T} \int_0^T (z_u(t) - z_r(t))^2 dt}$$

- the last index is taken from the standard ISO-2631 in a simplified way which measures comfort in function of the RMS of the acceleration of m_s , of course the lower the value the better the comfort:

$$\text{RMS}_{\ddot{z}_s} = \sqrt{\frac{1}{T} \int_0^T (\ddot{z}_s(t))^2 dt}$$

Thus, here's the scheme of the plant in SimuLink:

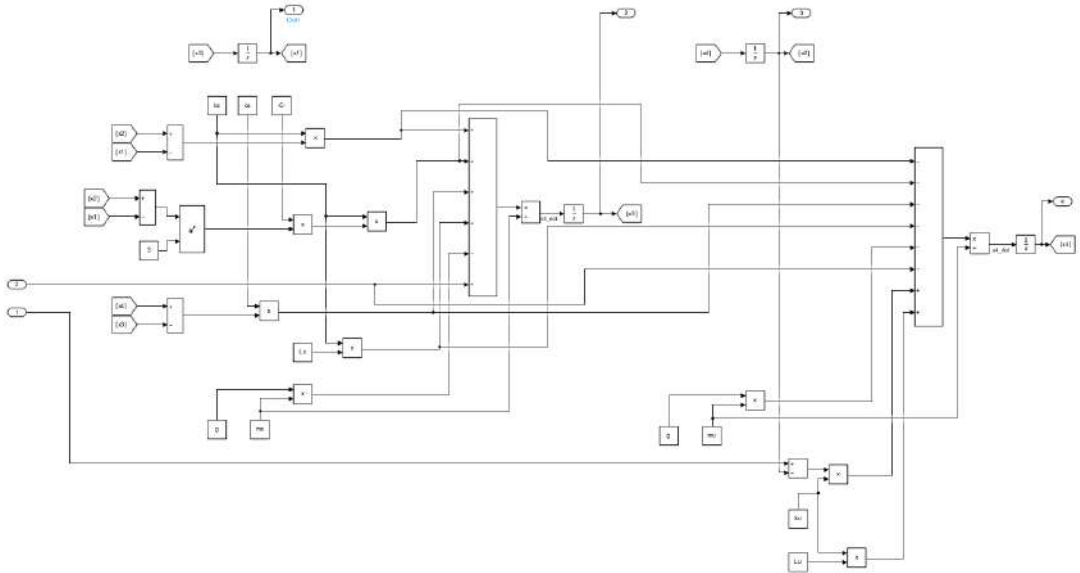


Figure 2.12: Quarter-Car Scheme

2.3.1 Road Bump

The first test is done on a road bump, those typically are quite annoying in terms of comfort even if the velocity is small and the road profile "simple" but it would be useful to see a basic and a complex scenario, so the inputs are:

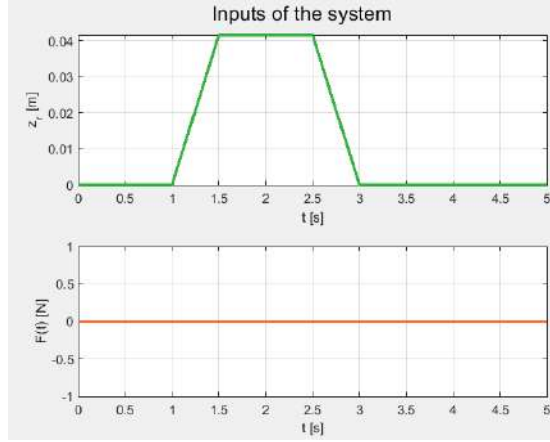


Figure 2.13: Inputs

And the response of the system is:

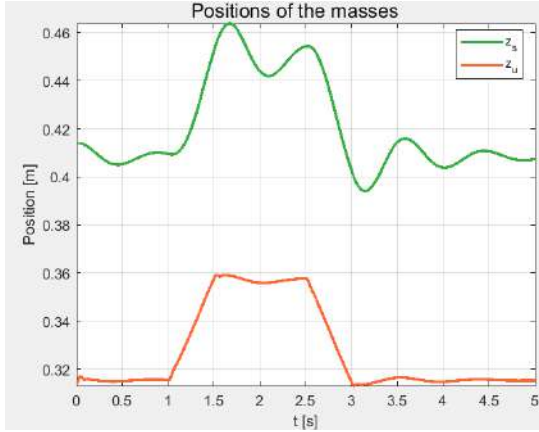


Figure 2.14: Positions

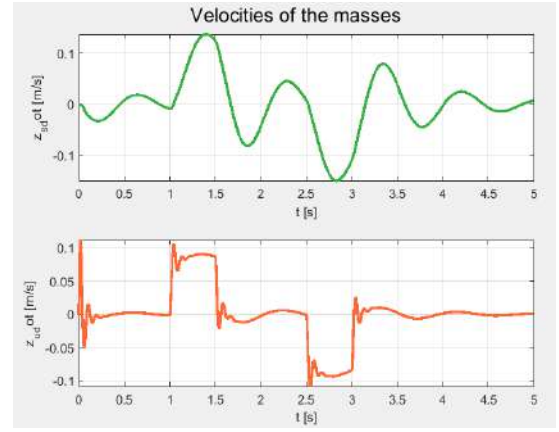


Figure 2.15: Velocities

We can notice that the behavior corresponds to the expected one, the sprung mass moves smoothly because it's not so damped, and in fact it oscillates slowly but far from the equilibrium point, then, the $RMS_{z_s} \approx 0.022$ m, but we can observe also that the maximum excursion is about 6.5cm and the $RMS_{\dot{z}_s} \approx 0.036$ m/s². Instead the unsprung mass moves rapidly and it's more damped, so it returns fast to it's equilibrium point such as to keep contact with the road. Hence, in this case $RMS_{u,r} \approx 0.32$ m which means that the wheel averagely is always in contact with the road, because we have to remember about the offset.

Table 2.1: Performance indices for passive suspension on road bump

Metric	Value
$\text{RMS}_{\dot{z}_s}$ [m]	0.022
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.36
$\text{RMS}_{u,r}$ [m]	0.32

2.3.2 Off-Road

The second road profile is much more challenging instead:

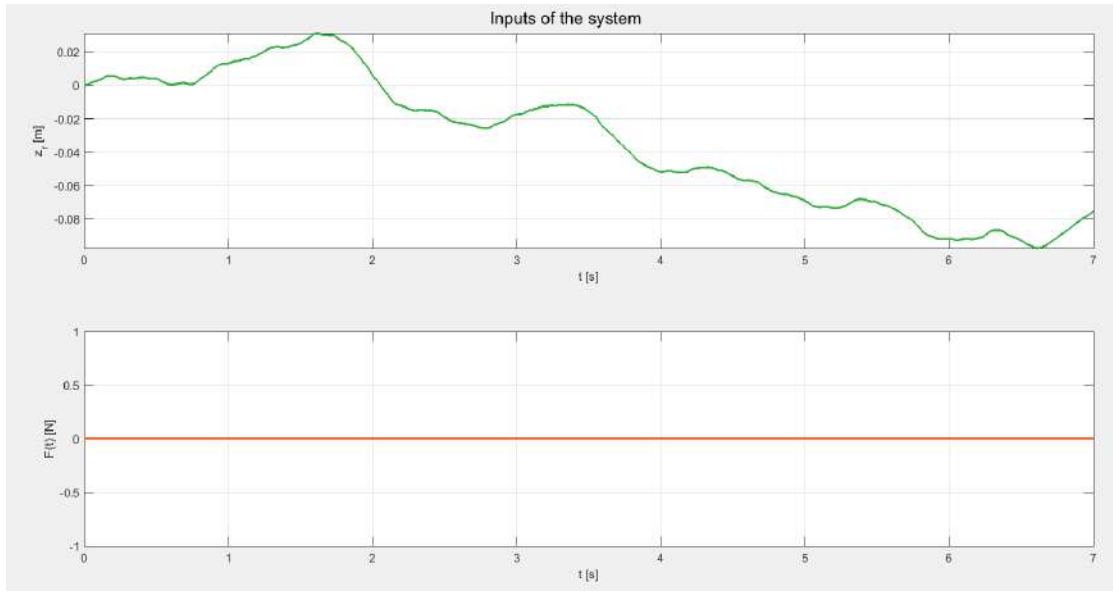


Figure 2.16: Inputs

And we can see that with the passive suspension configuration the positions are:

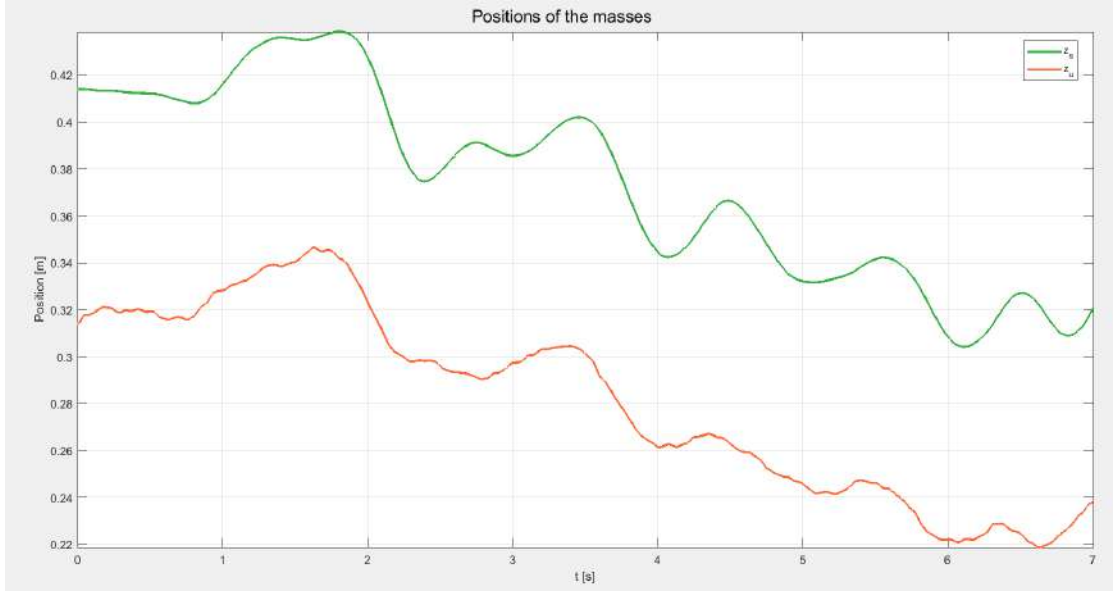


Figure 2.17: Positions

In this case, $RMS_{z_s} \approx 0.06 \text{ m}$ meaning that averagely the excursion of the car is around 6cm , road holding is implicit due to the parameters, as configuration of every generic car, $RMS_{u,r} \approx 0.32 \text{ m}$ and instead $RMS_{\ddot{z}_s} \approx 0.44 \text{ m/s}^2$. We'll use those parameters to evaluate the performance of the controllers implemented in chapter 3.

Table 2.2: Performance indices for passive suspension on off-road

Metric	Value
$RMS_{z_s} \text{ [m]}$	0.06
$RMS_{\ddot{z}_s} \text{ [m/s}^2\text{]}$	0.44
$RMS_{u,r} \text{ [m]}$	0.32

Here instead are the velocities and the block diagram of the Open-Loop scheme:

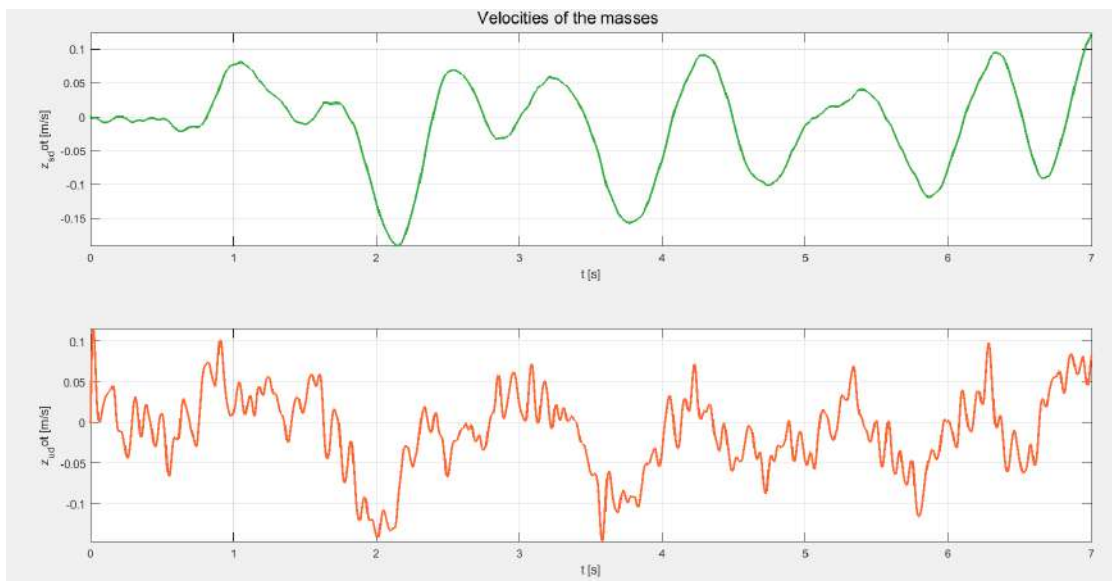


Figure 2.18: Velocities

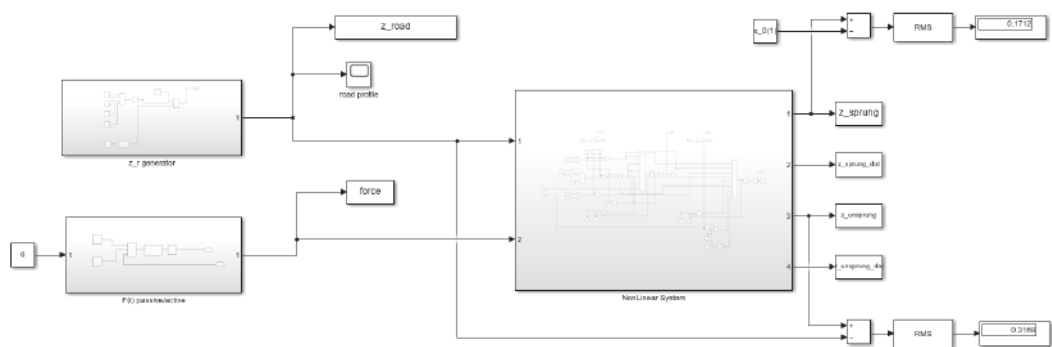


Figure 2.19: Block Diagram Scheme

Chapter 3

Synthesis

Now that the foundations are set we can focus on the crucial topic of the project, the synthesis of some controllers by which we'll try to balance comfort and road holding. We know also the behavior of the *passive* suspension system and this will help us to establish if the controllers are working fine. At the end we should be able to establish if the active suspension system is really worth it. After briefly settling down a useful control formulation for this case, we'll see how much an *LQR* scheme is capable of controlling the system, then we'll see *NonLinear MPC* in action, by which we can see the power of "being a few steps ahead" and not just simply to react.

3.1 Control Formulation

In the previous chapter, in particular section 2.3 we've indirectly introduced the performance indexes of the controllers, such as the RMS_{z_s} , $RMS_{u,r}$ and $RMS_{\ddot{z}_s}$. Because of course the primal aspects that are interesting in this case are the excursion of the car with respect to its default configuration (the equilibrium) and also we're interested in the tire's behavior, *independently* from the road profile z_r . But would those indexes be enough? Of course another important aspect will be the rapidity of the controller's response, this is a crucial part of the system and it should be real time (specially NLMPC). But those are slightly different topics from the ones of the course. Another relevant aspect is of course, how far should the initial conditions be from the equilibrium? Of course in normal conditions we won't find the car in the entire space R^n , either if we remember the MBC of Mercedes and its applications, they are implemented majorly for the city, and the active systems is used for random (and hopefully not so much) potholes, tram rails, bumps and so on. Basically in typical working conditions the values of z_s

and z_u will be almost equal to the equilibrium values. But for this case it will be enough since we well know that with positive damping the car will be stable, hence the system will be more interesting during the evolution of z_r because the Non-Linear spring will be felt more. In fact the second road profile was chosen to be challenging to demonstrate the system's potentialities. So to summarize, the control objectives consist in:

- minimizing the difference between the actual position of the car with respect to the equilibrium one, which means that we want the smaller possible $e_s = \bar{z}_s - z_s(t)$;
- we can say the same about the wheel, and considering the offset we would like to minimize $e_u = (z_r + \bar{z}_u) - z_u(t)$

3.2 LQR

The task to address now regards the implementation of a Linear controller which implementation is based on the *linearized* model but then we need to apply the control input to the original Non-Linear plant. Hence, we need to define the linearized system.

Of course the system will be evaluated near the only one possible equilibrium x_A and so we will evaluate the deviation of the state from the equilibrium values, $\tilde{x} = x_A + \delta_x$, thus:

$$\dot{\tilde{\mathbf{x}}} = A \tilde{\mathbf{x}} + B \mathbf{u} \quad (3.1)$$

While being:

$$A = \left. \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_A}, \quad B = \left. \frac{\partial g}{\partial \mathbf{u}} \right|_{\mathbf{x}=\mathbf{x}_A} \quad (3.2)$$

$$\tilde{\mathbf{y}} = C \tilde{\mathbf{x}}, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}. \quad (3.3)$$

This means that we already have A which is equal to the J of the system, we've seen it in section 2.1 and in particular its numerical form in picture 2.3. Now we only have to compute B , but let's consider its expression in the equation 1.14, we can see that it's already decoupled and independent from $\mathbf{x}$ and it just multiplies the vector of inputs $\mathbf{u}$, thus we can assume that $B = g(\mathbf{x}) = g$

If we want to implement the LQR controller we could want to check if our system is controllable, and we have to specify that the input z_r is of course not a real control input, so we'll need to check it only for the first column of the matrix B , since our system has dimension $n = 4$ the rank of the controllability matrix should assume the same value:

```

% Selecting the effective control input (active force F):
Bc = B(:,1);
Contr2 = ctrb(A, Bc);
disp("Rank of 'Real' Controllability Matrix :");
disp(rank(Contr2))

```

Figure 3.1: Script for the check of controllability in main.m

```

Rank of 'Real' Controllability Matrix :
4

```

Figure 3.2: Result

Before introducing the LQR technique we need also to mention that an integral action is needed to ensure that steady state error goes to zero and also to ensure robustness to model uncertainties that we'll introduce in the fourth chapter. So we need to augment our system and thus it will become a five dimensional system $n = 5$, thus:

$$\begin{aligned}\dot{x}_a &= \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} x_a + \begin{bmatrix} B \\ 0 \end{bmatrix} u + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r \\ y &= \begin{bmatrix} C & 0 \end{bmatrix} x_a\end{aligned}\quad (3.4)$$

since the state vector has become $x_a = [x_1, x_2, x_3, x_4, x_I]$, and the reference r will be zero.

At this point, we want to have an optimal control input:

$$u^* = -Kx = -K_x x - K_I x_I \quad (3.5)$$

Which needs to minimize the following quadratic function:

$$J = \int_0^\infty (x^\top Q x + u^\top R u) dt \quad (3.6)$$

Where Q is a positive semi-definite matrix which weights the states and R is a positive definite matrix that weights the control inputs. In this way we can add a bigger weight on states that we want to limit for example and of course we can do the opposite, same for the inputs of course. Regarding this, in order to define a way to choose the values of Q I've implemented the *Bryson's* criterion:

$$Q_{ii} = \frac{1}{x_{i,\max}^2}, \quad R_{jj} = \frac{1}{u_{j,\max}^2} \quad (3.7)$$

even if this is not a certain rule it's a good starting point from which by trial and error we can deduce the parameters by which we are satisfied:

```

% "accetable" limits
x1_max = 0.001; % m (position of the car; relevant)
x2_max = 1e3; % m (position of the wheel; not relevant)
x3_max = 0.08 ; % m/s (velocity of the car; relevant)
x4_max = 1e3; % m/s (velocity of the wheel; not relevant)
z_max = 0.1;

% Bryson
Q_bry = diag( 1./[x1_max^2, x2_max^2, x3_max^2, x4_max^2, z_max^2] );

Q = Q_bry;
Q(1,1) = 20 * Q(1,1);
Q(3,3) = 20 * Q(3,3);

% Weighting the Input Force
umax = 15000; % N
R0 = 1/(umax^2);

```

Figure 3.3: Bryson implemented in main.m in the LQR section

To obtain the matrix K we solve through MatLab the *Riccati's* algebraic equation:

```

% feedback gain vector
[Klq, P, e] = lqr( Aaug, Baug(:,1), Q, R );

Kx = Klq(1:4);
Ki = Klq(5);

```

Figure 3.4: Obtaining K in main.m

Before showing off the results we can evaluate if the Closed-Loop system is asymptotically stable or not, thus also if the augmented system is *stabilizable* and *detectable*:

```

Kx = Klq(1:4);
Ki = Klq(5);

A_cl = Aaug - Baug*Klq;
lambda_cl = eig(A_cl);
disp('C-L Eigenvalues:')
disp(lambda_cl)

if all(real(lambda_cl) < 0)
    disp('C-L system is stable');
else
    disp('C-L system is not stable');
end

```

Figure 3.5: Checking for stability in main.m

```

C-L Eigenvalues:
-19.6695 +72.9702i
-19.6695 -72.9702i
-42.9565 +34.6162i
-42.9565 -34.6162i
-0.0447 + 0.0000i

C-L system is stable

```

Figure 3.6: Result

3.2.1 Road Bump

Let's see the controller in action on the classical bump starting not in the equilibrium and applying the control action on the Non-Linear plant:

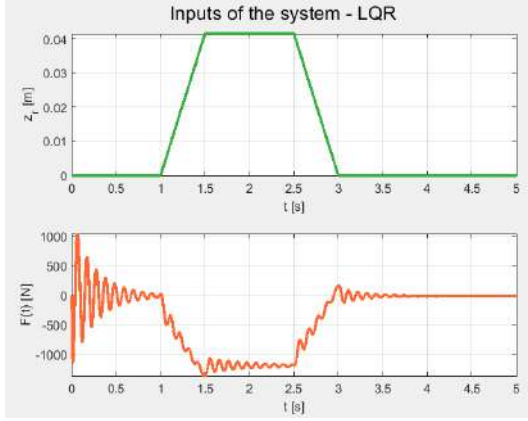


Figure 3.7: Inputs

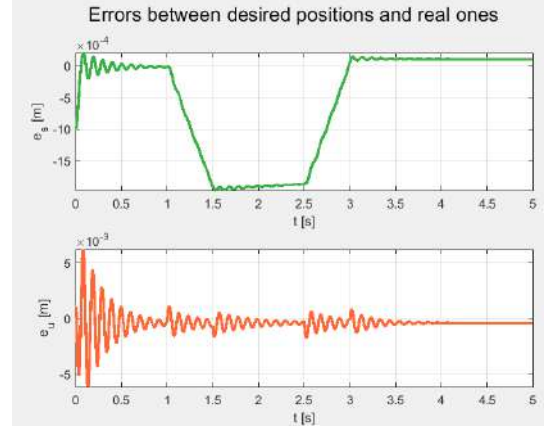


Figure 3.8: Errors

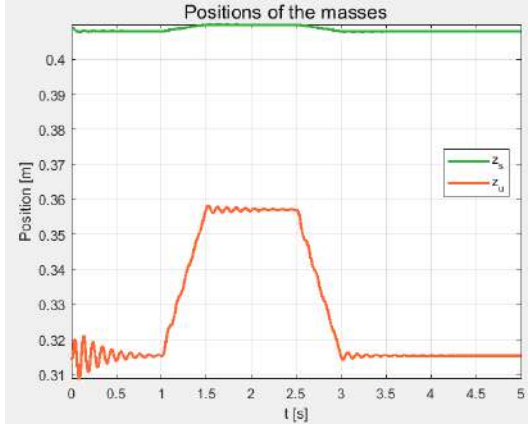


Figure 3.9: Positions

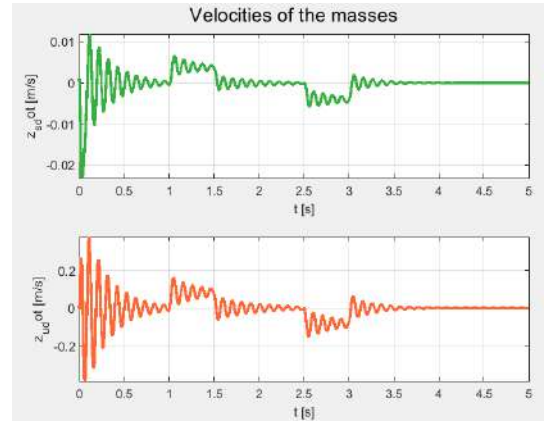


Figure 3.10: Velocities

We can see that the results are quite stunning, from an excursion of the "disturbance" of about 4cm the car moves off by millimeters, and instead the tire is following nicely the road, in fact the 2 errors are almost zero practically. Let's wrap up the results in the following table:

Table 3.1: Performance indices for active suspension on road bump LQR

Metric	Value
$\text{RMS}_{z_s} \text{ [m]}$	0.001
$\text{RMS}_{\ddot{z}_s} \text{ [m/s}^2\text{]}$	0.17
$\text{RMS}_{u,r} \text{ [m]}$	0.32

3.2.2 Off-Road

In this case instead:



Figure 3.11: Inputs

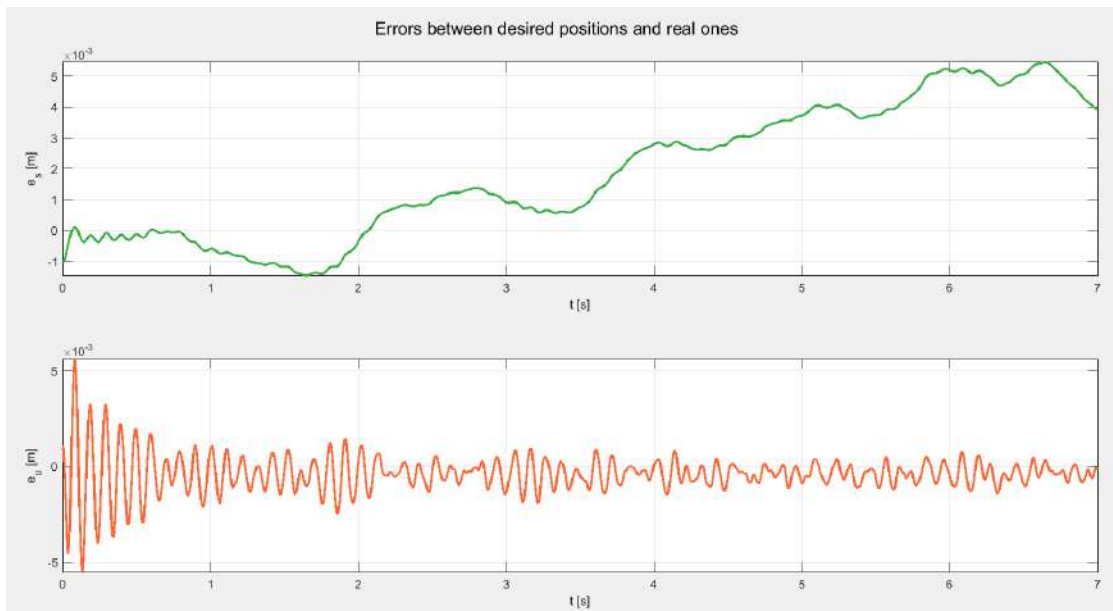


Figure 3.12: Errors

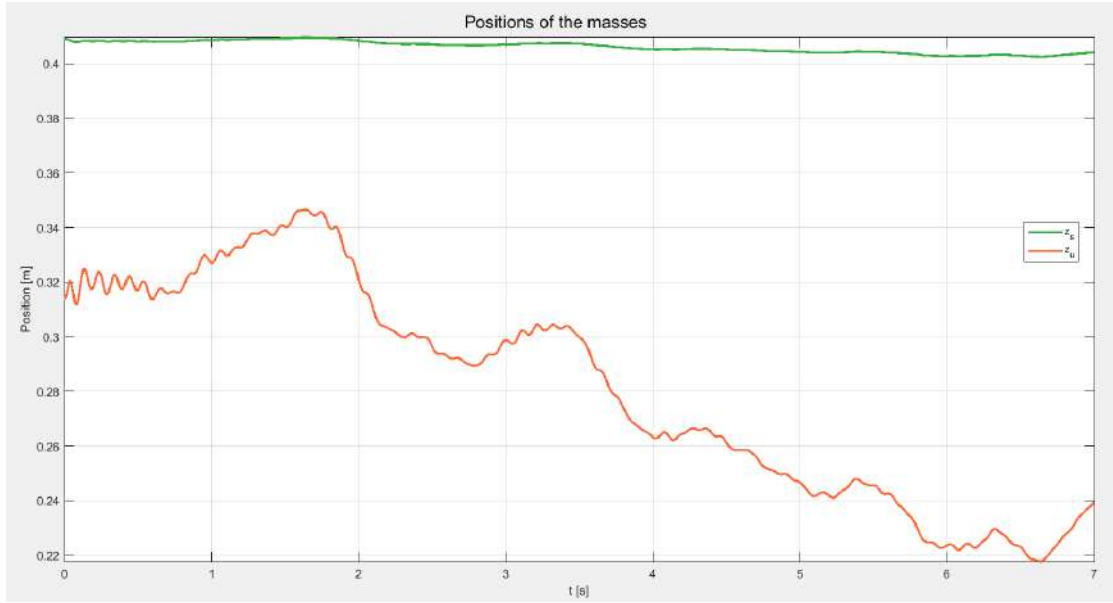


Figure 3.13: Positions

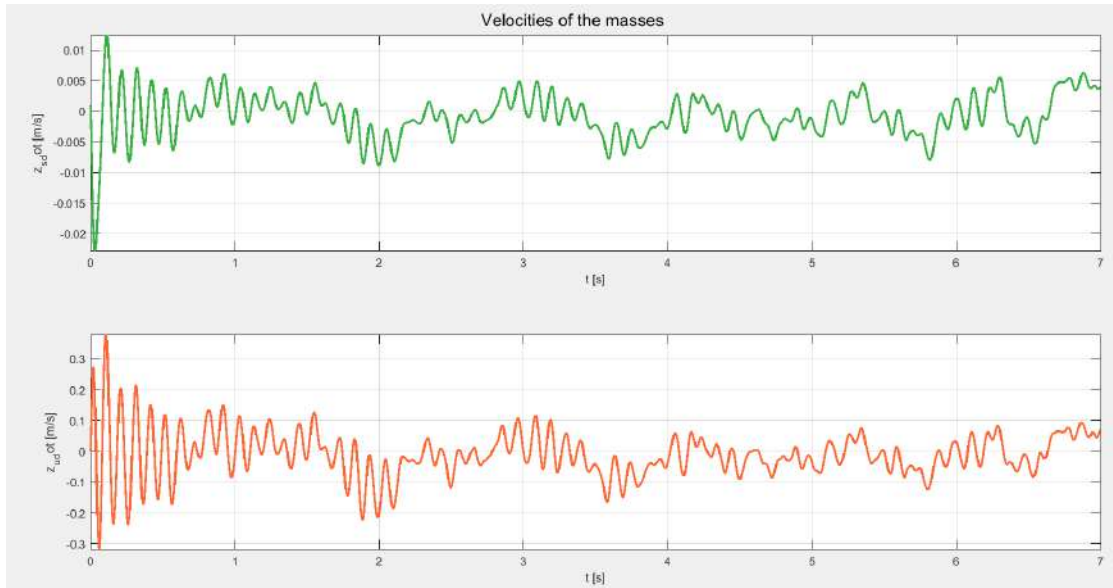


Figure 3.14: Velocities

As we can notice in this case instead, the errors are still of the order of millimeters and its quite good considering also the road conditions and the fact that its maximum excursion is of about 10 cm. The parameters as we can see are quite good even now compared to the ones of Table 2.2:

Table 3.2: Performance indices for active suspension on off-road LQR

Metric	Value
$\text{RMS}_{\ddot{z}_s}$ [m]	0.003
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.15
$\text{RMS}_{u,r}$ [m]	0.32

Here's instead the used scheme, NonLinear plant plus Linear controller:

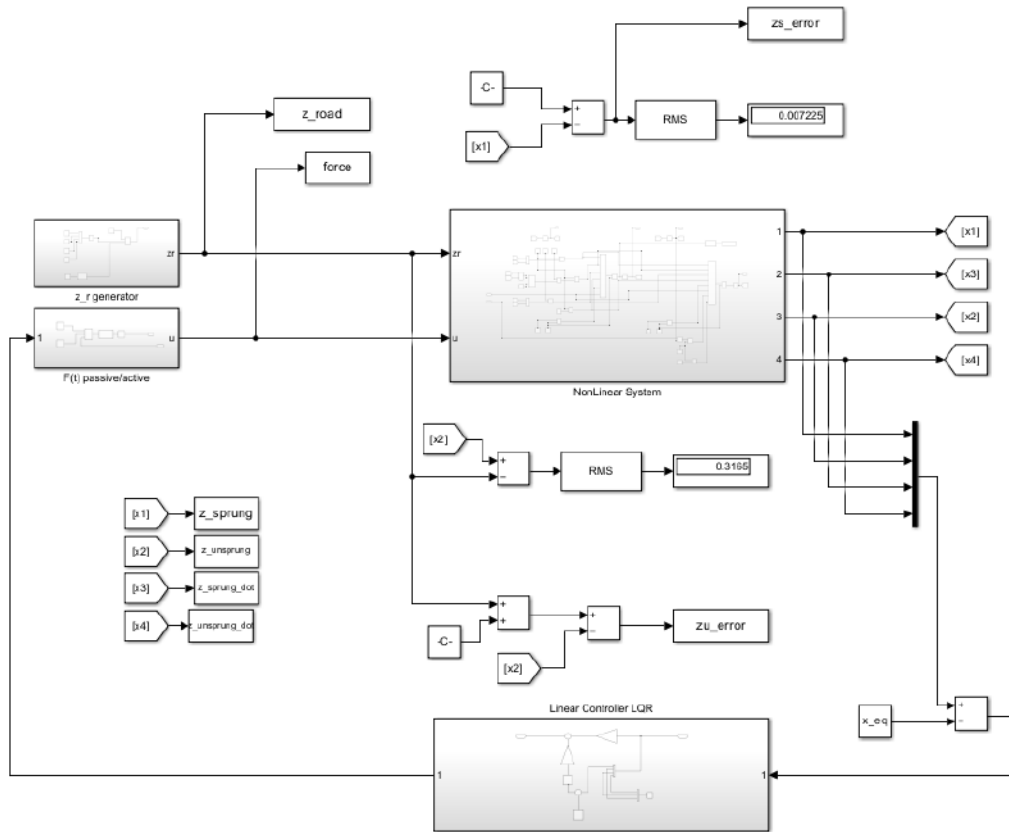


Figure 3.15: Block Scheme

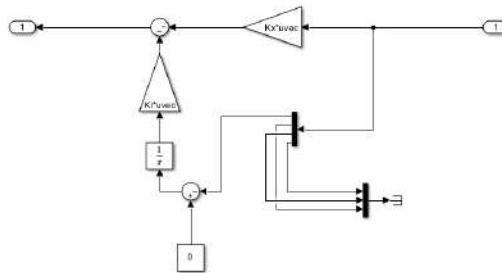


Figure 3.16: LQR Controller

3.3 NL MPC

Now, finally has come the time to consider some more advanced control schemes, and in particular my only choice is the Non Linear Model Predictive Control. So the questions that I want to address now is: "Why only MPC?" I've found the answer during my researches on this argument for my project and in particular thanks to [6]. In that article they've discussed about different ways of controlling an active suspension system, via *LQR*, *sliding mode* and so on, but they were fascinated by the power of *MPC* and its capabilities thanks to the model to "predict the future" and let me say, play with the problems, discovering them and by preventing them. It's even more powerful considering it combined to the MBC as we'll see.

In particular *MPC* can work also in real time and they even have made the problem simpler by introducing the *flat outputs* modelation, by which they re-organize the problem as linear one, making the solution even more rapid.

In my case instead I've decided to implement MPC around the NL model and see what happens. And in particular I've implemented through the MatLab's *nlmpc* object¹. So in order to introduce a little bit the MPC I'll combine theory and practical things.

We can consider MPC as a chess player, because starting from a certain instant of time, it decides a certain control action $u(k)$ and through the model of the plant it can predict what are the output in terms of the system's behavior, then it does it another $n - 1$ times if we consider n as the prediction horizon. After all of this it selects the first input and goes to the next instant of time $k + 1$ and repeats the process. In this way the controller can choose wisely the control sequence U^* by which it tries to minimize an objective function J defined as the one from *LQR* and by respecting some constraints on the states and inputs defined by us.

So in the definition of an *nlmpc* object of course we'll need to define the number of states, inputs (by considering the active ones and measured disturbances) and output, which correspond to the variables we want to control in a certain way:

¹<https://it.mathworks.com/help/mpc/ref/nlmpc.html>

```

nx = 4;
ny = 1;

mvIndex = 1;
mdIndex = 2;

nlobj = nlmpc(nx, ny, 'MV', mvIndex, 'MD', mdIndex);
nlobj.Dimensions

nlobj.Ts = ts;
nlobj.Model.IsContinuousTime = true;
nlobj.Model.StateFcn = 'quarterCarStates';

% OutputFcn
nlobj.Model.OutputFcn = @(x,u) [
    x(1)
];

```

Figure 3.17: MPC Init

As we said we can impose also some desired constraints on our state variables:

```

% x1 = z_s car position
nlobj.States(1) = struct('Min', 0.408-0.002, 'Max', 0.408+0.002, ...
    'Name', 'z_s', 'Units', 'm', 'ScaleFactor', 0.002);

% x2 = z_u wheel position
nlobj.States(2) = struct('Min', 0.315-1, 'Max', 0.315+1, ...
    'Name', 'z_u', 'Units', 'm', 'ScaleFactor', 1);

% x3 = dz_s car's velocity
nlobj.States(3) = struct('Min', -3, 'Max', 3, ...
    'Name', 'dz_s', 'Units', 'm/s', 'ScaleFactor', 3);

% x4 = dz_u wheel's velocity
nlobj.States(4) = struct('Min', -10, 'Max', 10, ...
    'Name', 'dz_u', 'Units', 'm/s', 'ScaleFactor', 10);

```

Figure 3.18: MPC states constraints

and of course the MPC has to take in account about the actuator limits, even talking for example its rate of change:

```

Fdot_max = 50000;      % N/s realistic
Rate_limit = Fdot_max *ts;

nlobj.MV(1) = struct( ...
    'Min', -15000, 'Max', 15000, ...
    'MinECR', 0, 'MaxECR', 0, ...
    'RateMin', -Rate_limit, ...
    'RateMax', Rate_limit, ...
    'RateMinECR', 0, 'RateMaxECR', 0, ...
    'Name', 'F_attiva', 'Units', 'N', ...
    'ScaleFactor', 15000 ...
);

```

Figure 3.19: MPC actuator limits

Though the above lines of code it can automatically chose the weight of the Q and R matrices in the objective function and that's one of the nice things of this class of object. But there are also other interesting things, for example I've decided to accept also sub-optimal solutions and very usefully I've chosen to interpolate the sequence of discrete optimal inputs $u(k)^*$, plus the addition of slack variables on the constraints:

```

% OutputVariables weight
nlobj.Weights.OutputVariables = 2; % [z_s]

% MV → active force
nlobj.Weights.ManipulatedVariables = 0.01;
nlobj.Weights.ManipulatedVariablesRate = 0.1;

% Slack variables
nlobj.Weights.ECR = 1e2;

nlobj.Optimization.MVInterpolationOrder = 1;

nlobj.Optimization.UseSuboptimalSolution = true;

```

Figure 3.20: MPC extra features

And thanks to the *nlmpcmove* function we can easily simulate a run of the prediction in a certain instant of time:

```

for k = 1:Nsteps

    % z_r at the current instant
    md = z_r_disc(k);

    % computing optimal input
    mv = nlmpcmove(nlobj, x, lastmv, yref, md);

    u_full = [mv; md];

    % simulating the system
    dyn = @(t,x) quarterCarStates(x, [mv; md]);
    [~, x_temp] = ode45(dyn, [0 ts], x);

    % take the last value
    x = x_temp(end,:);

    % save the data
    xHistory = [xHistory x];
    mvHistory = [mvHistory mv];

    lastmv = mv;
end

```

Figure 3.21: MPC Main Loop

3.3.1 Road Bump

So without losing time and by just specifying that the reference for z_s was set without approximation up to $\bar{z}_s = 0.408m$

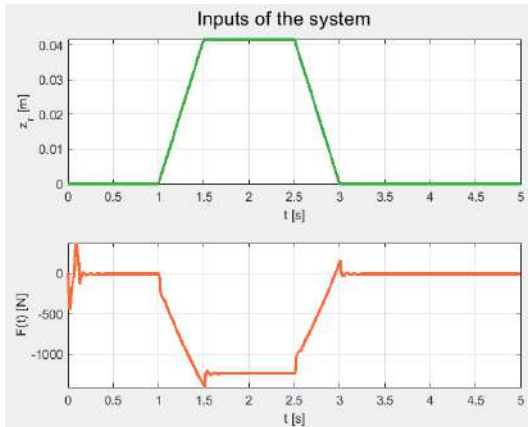


Figure 3.22: Inputs

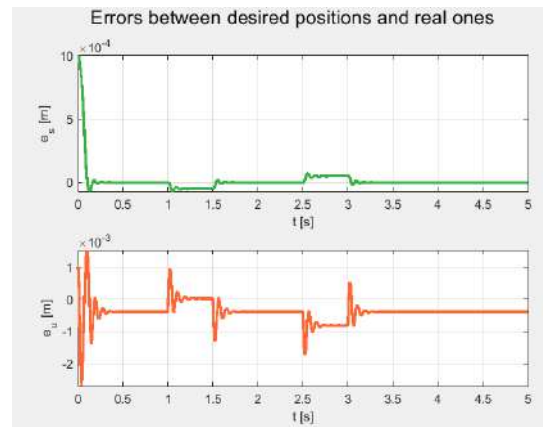


Figure 3.23: Errors

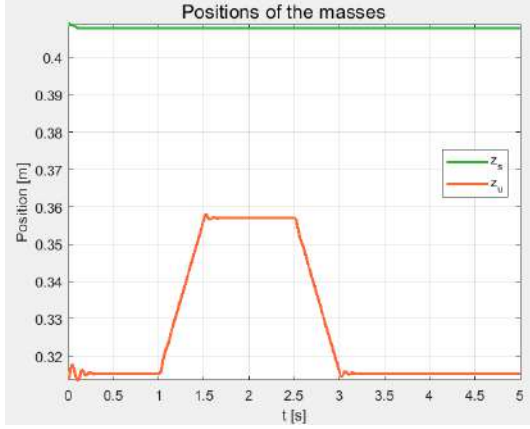


Figure 3.24: Positions

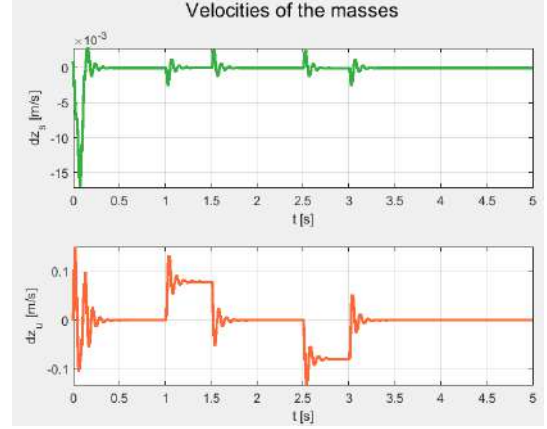


Figure 3.25: Velocities

And it's even not the best part, so let's wrap up rapidly the table:

Table 3.3: Performance indices for active suspension on road bump NLMPC

Metric	Value
$\text{RMS}_{z_s} \text{ [m]}$	0.001
$\text{RMS}_{\ddot{z}_s} \text{ [m/s}^2\text{]}$	0.05
$\text{RMS}_{u,r} \text{ [m]}$	0.32

3.3.2 Off-Road

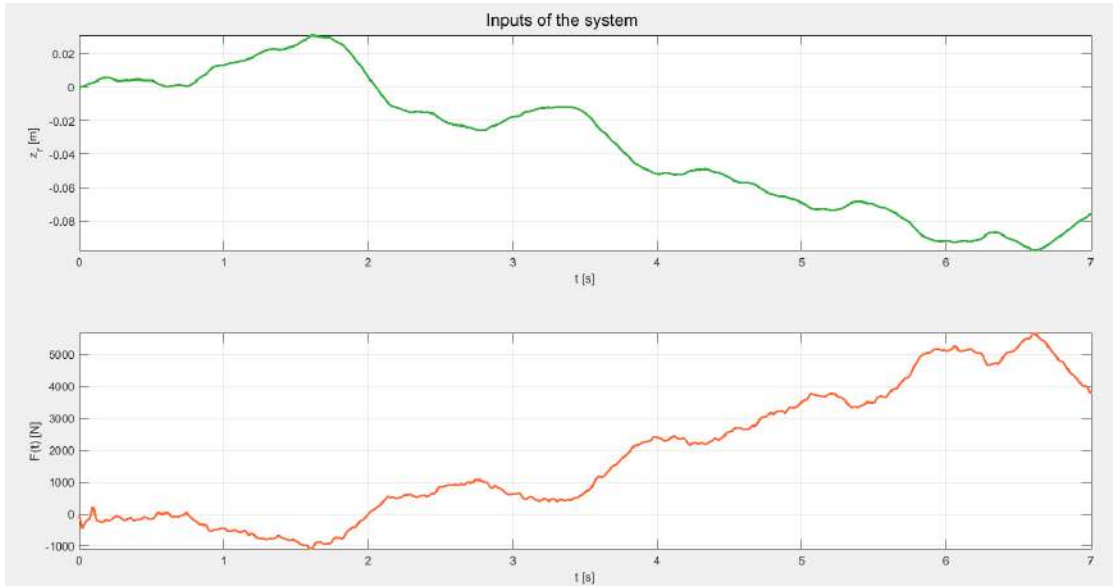


Figure 3.26: Inputs

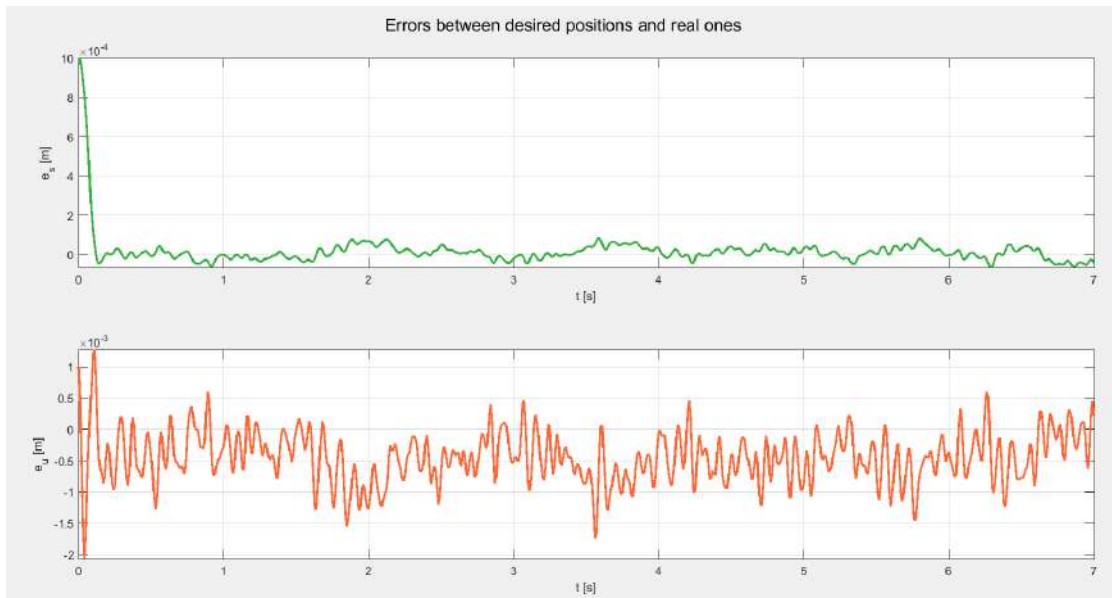


Figure 3.27: Errors

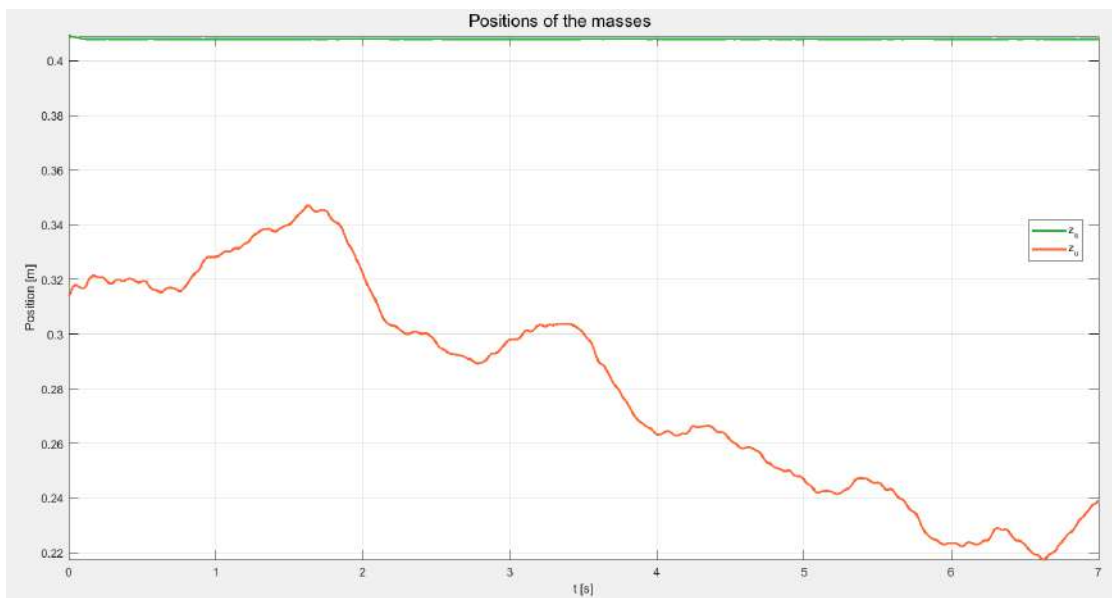


Figure 3.28: Positions

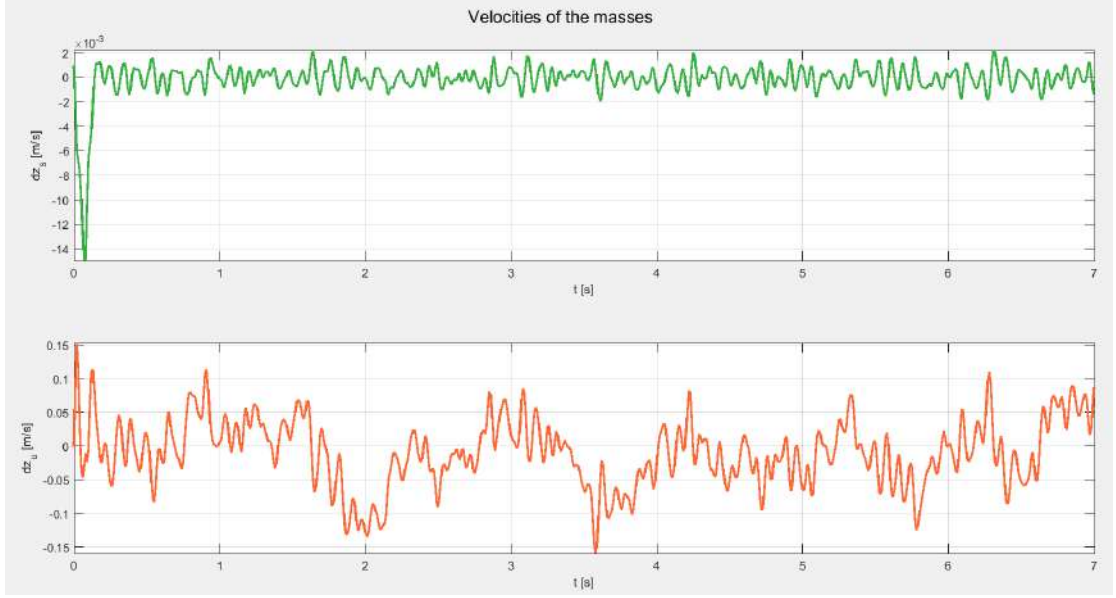


Figure 3.29: Velocities

I think that the results are quite stunning, besides it speaks by itself let's make some considerations. Without even getting close to the saturation of the actuator and by neither increasing $F(t)$ sensitively with respect to the LQR control, we can see that the car sees nothing about the road conditions. For sure it will not work everywhere, there will always be some limits, impossible road conditions which will of course sacrifice comfort. However in this case though MPC we've archived both comfort and road holding.

The general discussion will be taken later on, so now let's wrap up our references for the performance, in the next chapter we'll evaluate the controllers under even worse conditions:

Table 3.4: Performance indices for active suspension on off-road NLMPC

Metric	Value
RMS_{z_s} [m]	0.0001
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.06
$\text{RMS}_{u,r}$ [m]	0.32

3.4 Extra, Linear Spring

I think that some useful thing to show you is the effects of the Non-Linearity of the spring, in fact if for example we set $\epsilon = 0$ we will model a simple linear spring and in off road conditions the results will be instead:

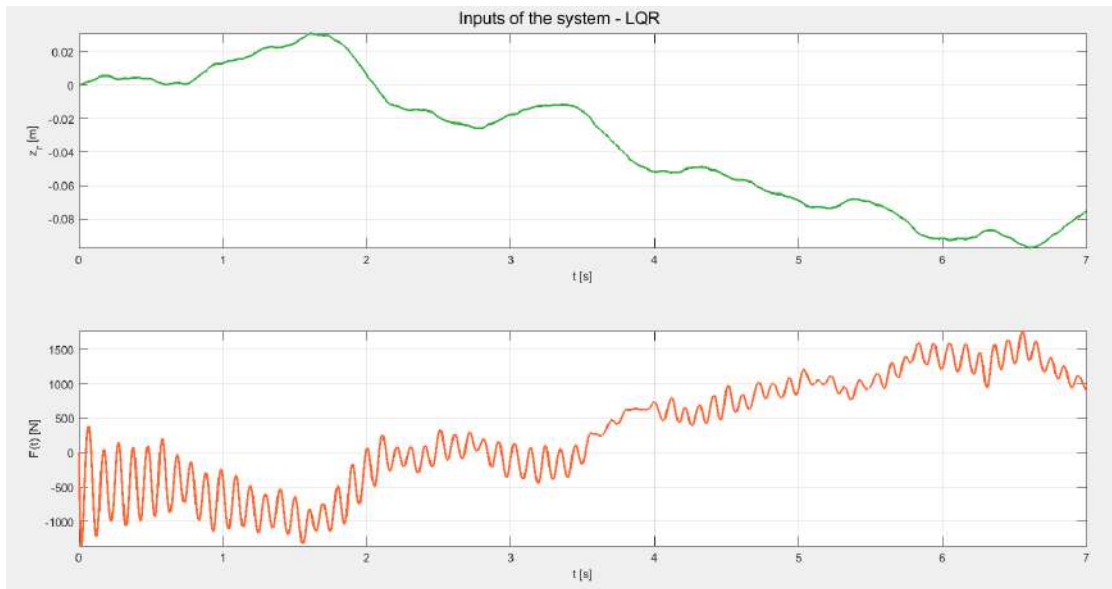


Figure 3.30: Inputs

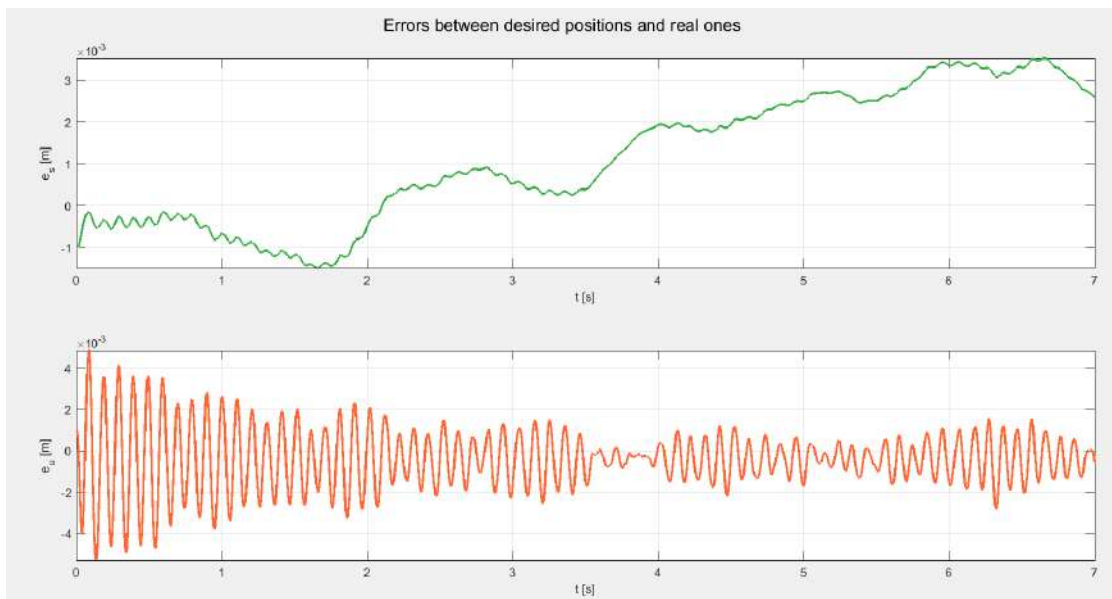


Figure 3.31: Errors

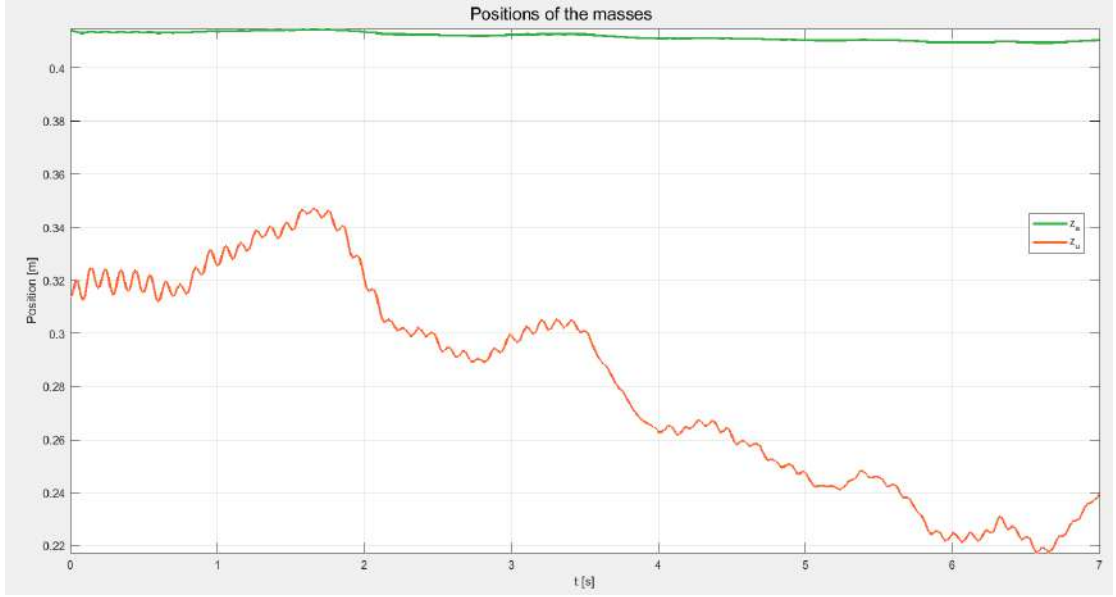


Figure 3.32: Positions

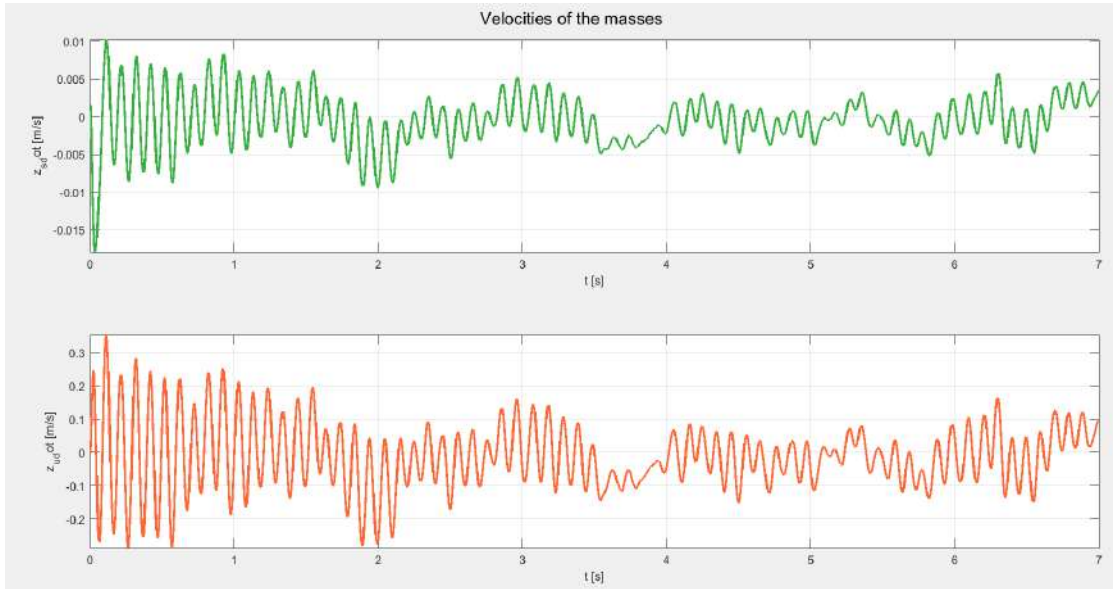


Figure 3.33: Velocities

And from this test there are a few things that we can notice, the first thing is that of course the required force is about 5 times less than the previous one, and it's quite obvious because we're modeling the hardening of a spring with ϵ , so if it's zero, for bigger excursions of it the force will not increase also by cubic law and it's a good thing, the reason is that for sure the actuators will have some physical limits (assumed at $15kN$ in this project) and if we will have bigger excursions maybe at a certain point the piston will not be able to compensate for the road.

Maybe it's a drawback that the suspension becomes stiffer with excursion (cubically), but from a different point of view it's a plus regarding the *progressive* suspensions. It's like they can face easily some difficult road conditions and guarantee better performances, because without ϵ the system has bigger oscillations as we've noticed.

The simulation was done with LQR, and the differences with respect to NLMPC is that the oscillations are mitigated.

Chapter 4

Robustness Analysis

In this chapter we'll focus on the robustness of the implemented controllers, since there's the need to perform tests on parametric uncertainty and disturbance rejection I've thought about interesting and maybe realistic ways of testing those scenarios and we'll discuss the performance of each controller for this different types of problems.

4.1 Parametric Uncertainties

At the beginning, one of my personal questions was: "What parameter or parameters I will assume to be uncertain?" And immediately I've thought about the fact that we're not considering something relevant in the parameters. Because for sure we consider the quarter of the total car's mass, however we're not considering the people sitting in it. Since we're considering a single wheel it straight forward to consider the contribution of the single person sitting near each wheel/suspension. To model a general case we will consider an additional unmodeled mass of $70kg$ acting on m_s . Then let's consider for example that the owner of the car enjoys some after-market customizations, one of the most beautiful ways to chance a car's look and performance is to change the wheels. Thus we'll consider that the m_u is lower then expected, for example of $7kg$ so let's see what happens in those cases:

4.1.1 Road Bump

We'll start from LQR, so let's see the results:

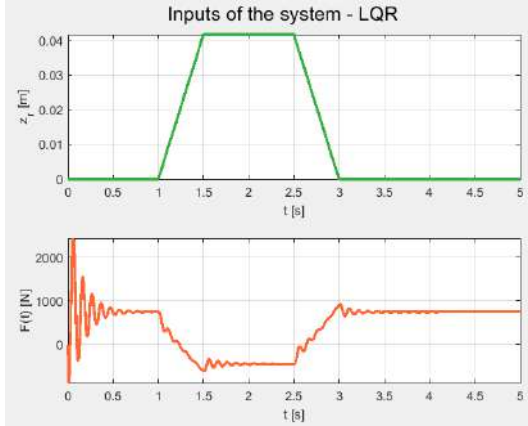


Figure 4.1: Inputs

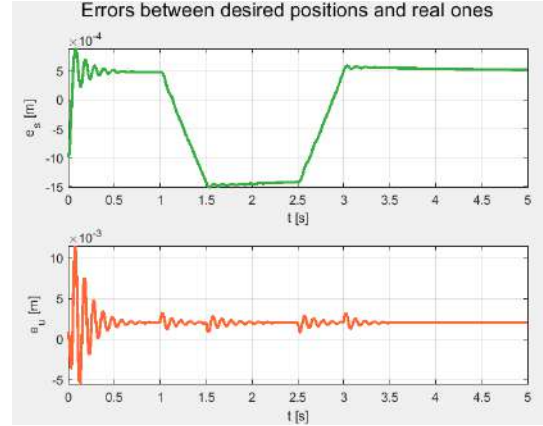


Figure 4.2: Errors

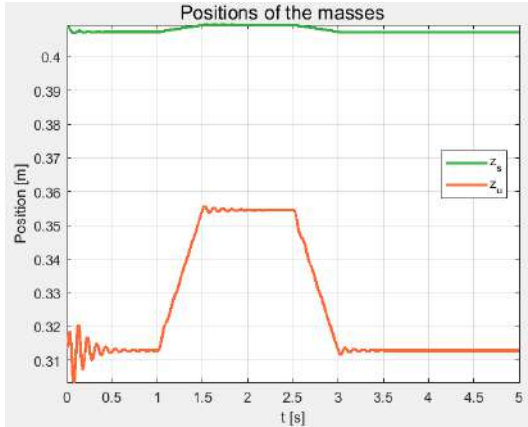


Figure 4.3: Positions

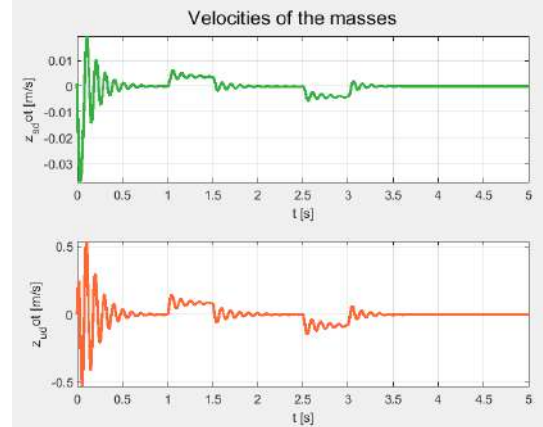


Figure 4.4: Velocities

At a first glance we would say that the behavior it's the same as before but if we look better at the differences by considering pictures from 3.7-3.10 there's something that need some discussion.

It's clear that the actuation force is bigger then before, and the answer is physical: with our control system we're trying to "lift" the car and let the wheel be compliant to z_r . If the mass of the vehicle is bigger then expected then a bigger force will be required to lift the vehicle.

Another aspect regards static conditions, we can see that without uncertainties the lifting force is almost zero, maybe there will be a residual small value of $F(t)$ due to uncertainties but is negligible. (figure 3.7), When the mass is bigger, the natural equilibrium of the car will be lower:

Equilibrium points:			
x1	x2	x3	x4
—	—	—	—
0.39	0.31	0	0

Figure 4.5: Values of the equilibrium with uncertainty on parameters

But the controller doesn't know it, so it will tend to *lift up* the system from "falling" to the new equilibrium. Thus we can say that the system works well in this case but if implemented in a real scenario there will be the need to consider an average mass of a person because if the active force will need to be constantly equal to almost $1kN$ in static condition instead of zero, this can lead to the shortening of the piston's life.

Some other relevant information we can retrieve from the simulation regards velocity oscillations, since the sprung mass is bigger then the system will have a larger inertia, thus the oscillations will be a little lower.

Table 4.1: Performance indices for active suspension on road bump with uncertainty LQR

Metric	Value
RMS_{z_s} [m]	0.0008
$RMS_{\ddot{z}_s}$ [m/s ²]	0.3
$RMS_{u,r}$ [m]	0.32

Now it's the turn instead for NLMPCC:

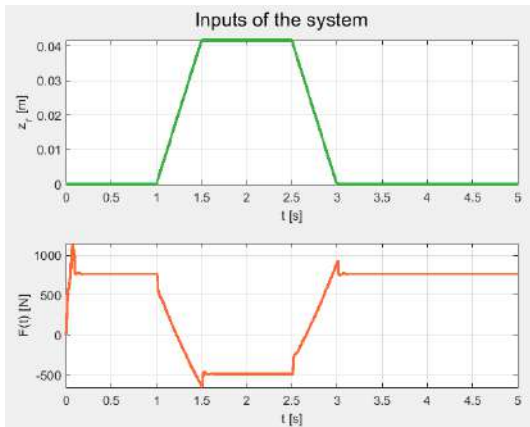


Figure 4.6: Inputs



Figure 4.7: Errors

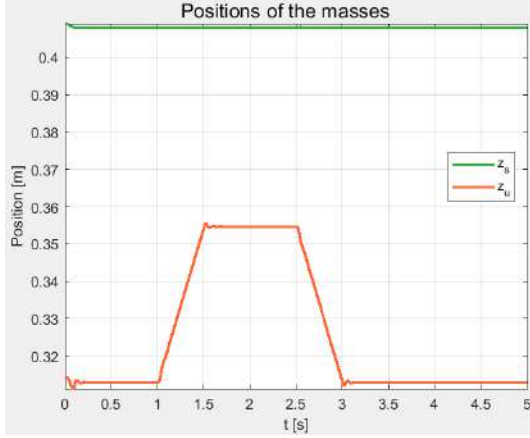


Figure 4.8: Positions

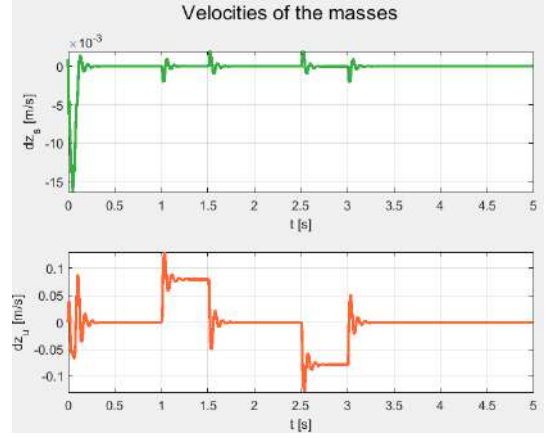


Figure 4.9: Velocities

And we can notice the power of NLMPC, if considering the force $F(t)$ in the picture 4.1 we can state that the equilibrium force of persists, but it's lower respect to LQR. Thanks to the prediction and optimality of the controller the force has a peak value and oscillations that are both lower in this case. Regarding positions, LQR works pretty fine but NLMPC is even better and it limits also the velocities, which is of course a relevant aspect:

Table 4.2: Performance indices for active suspension on road bump with uncertainty NLMPC

Metric	Value
$\text{RMS}_{z_s} [\text{m}]$	0.0001
$\text{RMS}_{\ddot{z}_s} [\text{m/s}^2]$	0.06
$\text{RMS}_{u,r} [\text{m}]$	0.32

4.1.2 Off-Road

In this other scenario the statements of the previous one continue to be valid, so let's just look at the simulations and the tables.

Let's start with LQR:

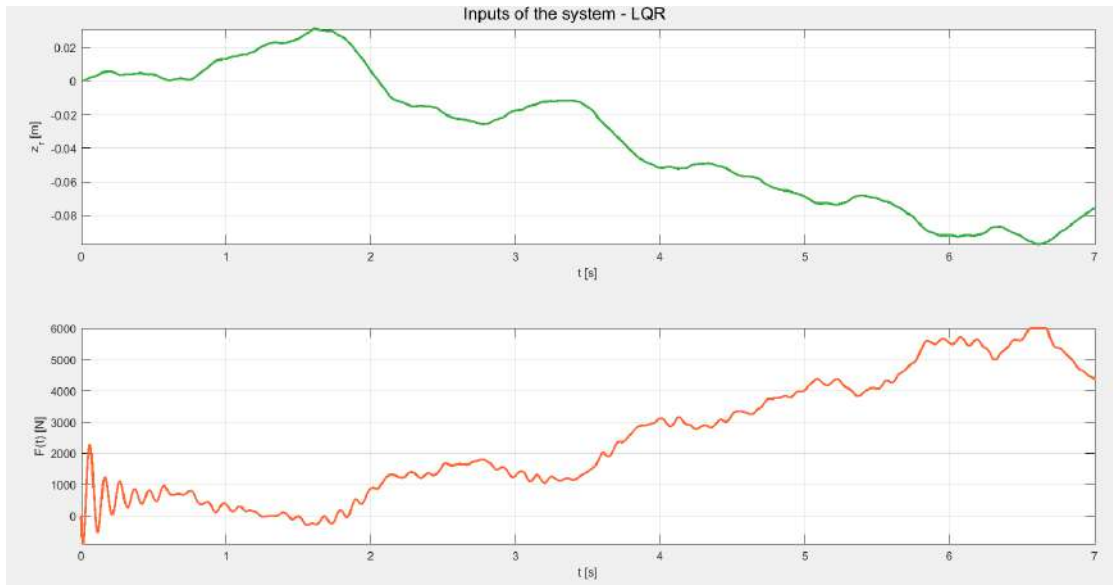


Figure 4.10: Inputs

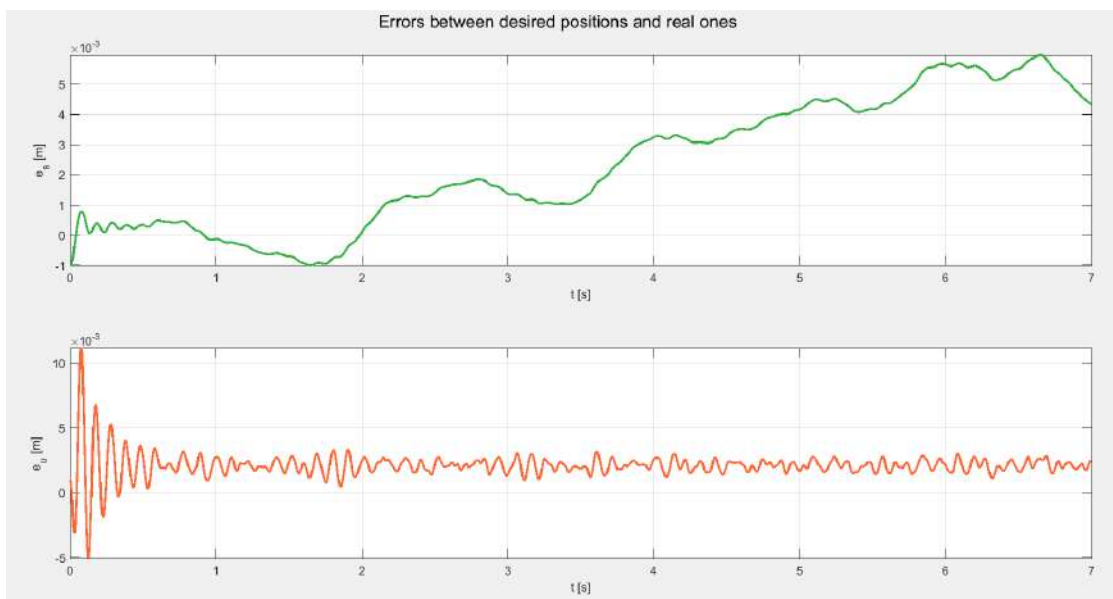


Figure 4.11: Errors

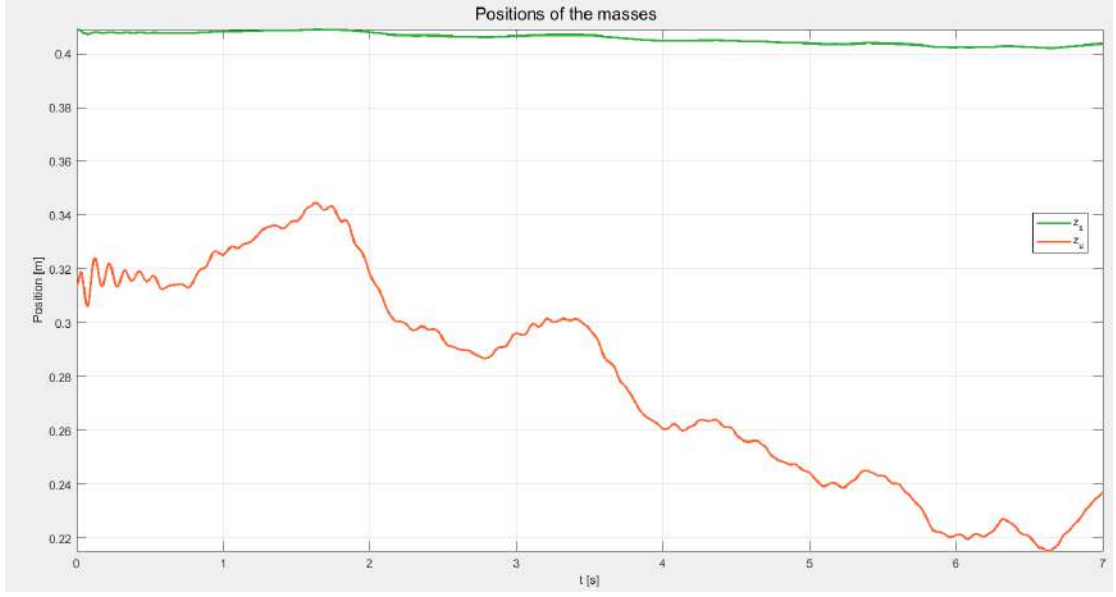


Figure 4.12: Positions

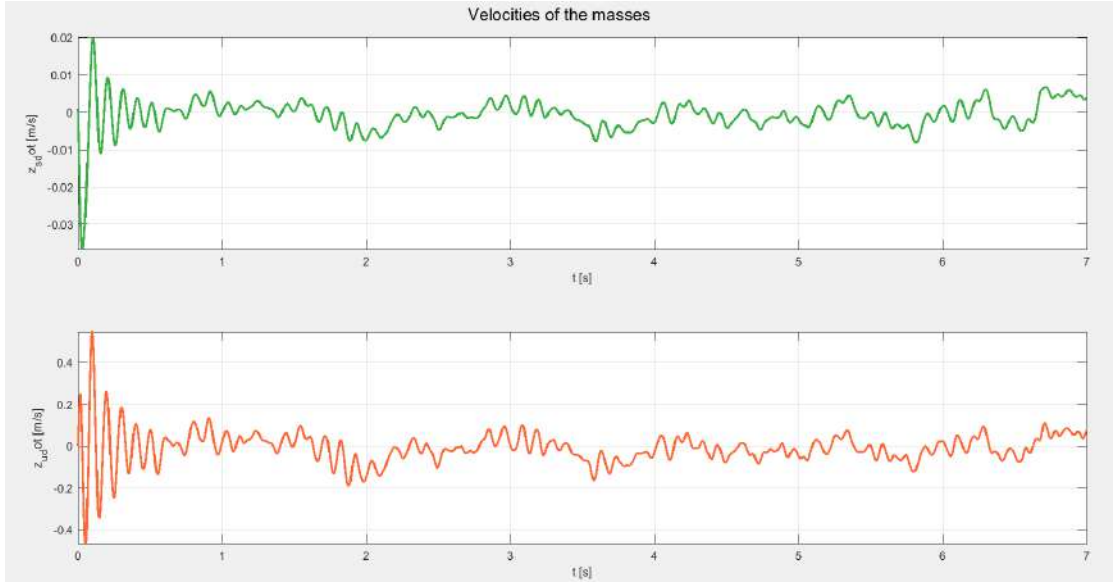


Figure 4.13: Velocities

Table 4.3: Performance indices for active suspension on off-road with uncertainty LQR

Metric	Value
RMS_{z_s} [m]	0.003
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.23
$\text{RMS}_{u,r}$ [m]	0.32

and then NL MPC:

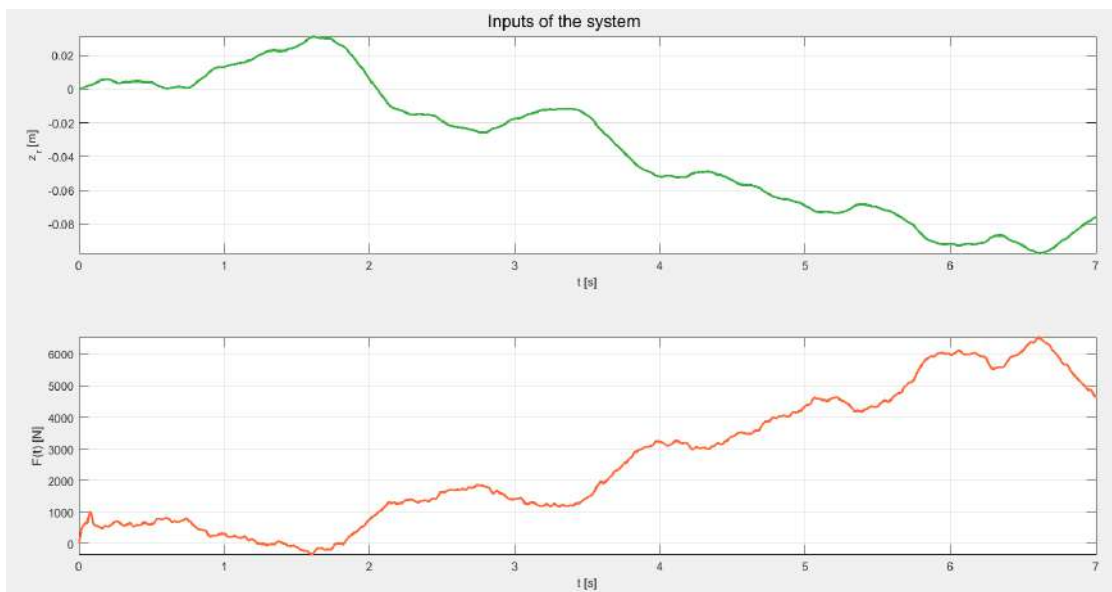


Figure 4.14: Inputs

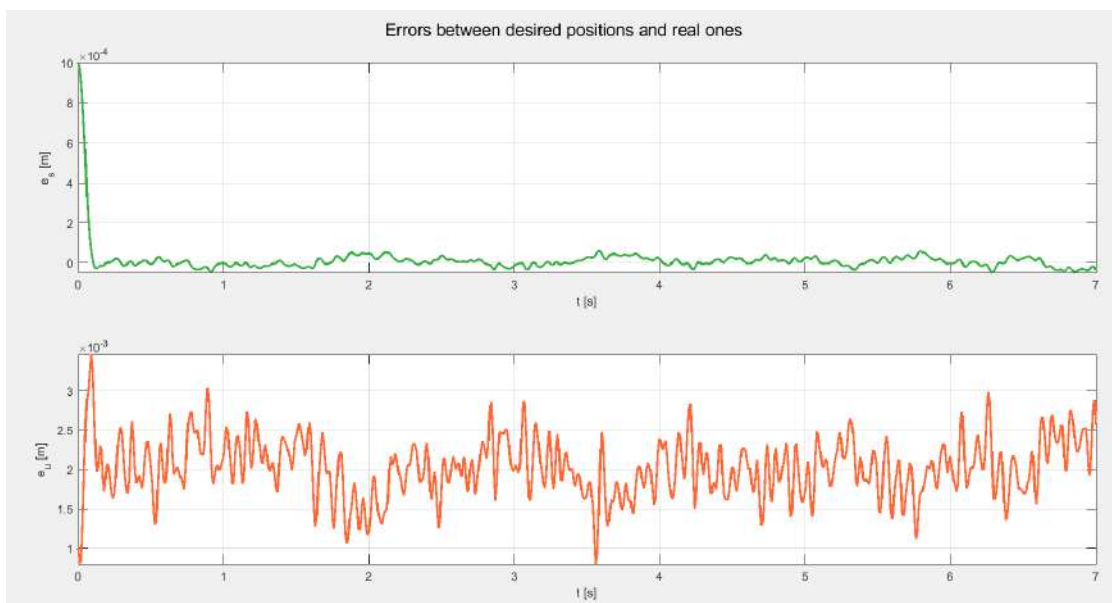


Figure 4.15: Errors

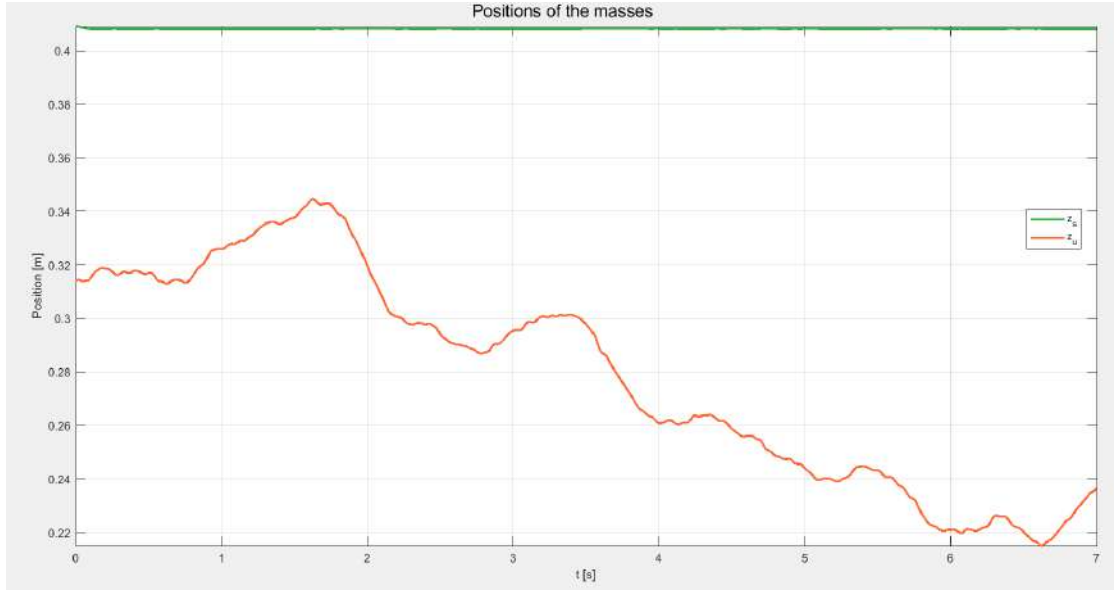


Figure 4.16: Positions

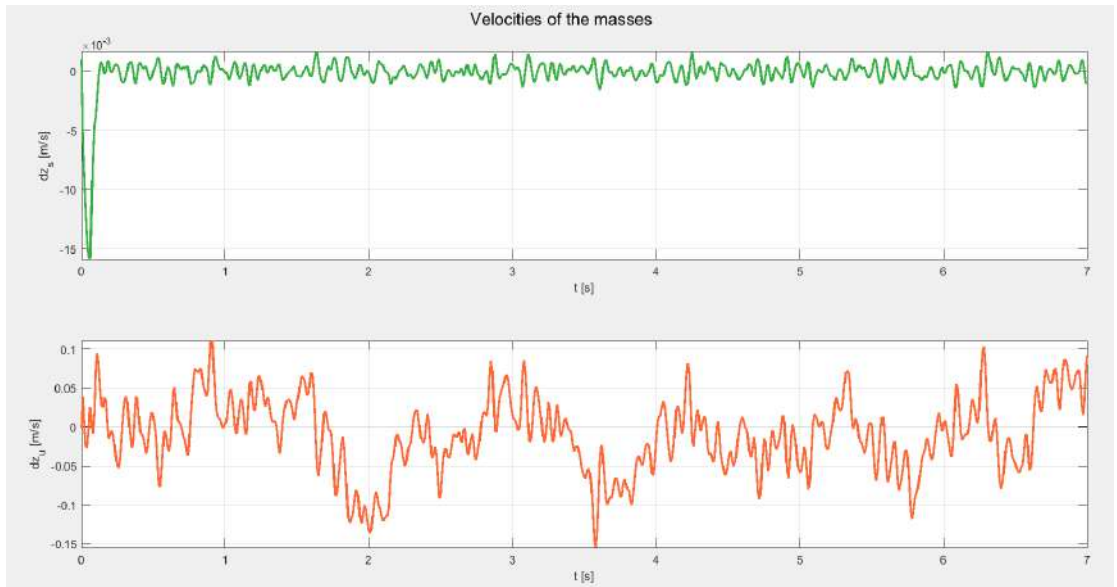


Figure 4.17: Velocities

Table 4.4: Performance indices for active suspension on off-road with uncertainty NLMPC

Metric	Value
$\text{RMS}_{\dot{z}_s}$ [m]	0.0001
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.06
$\text{RMS}_{u,r}$ [m]	0.32

4.2 Disturbance

Regarding the choice of the disturbance acting on the system, I've instantly thought about the MBC and on the fact that the system's precision to estimate the z_r has a precision of about $3mm$ thus I've decided to consider the noise of sensors and errors on the estimation on the input signal z_r , thus, the noise will be:

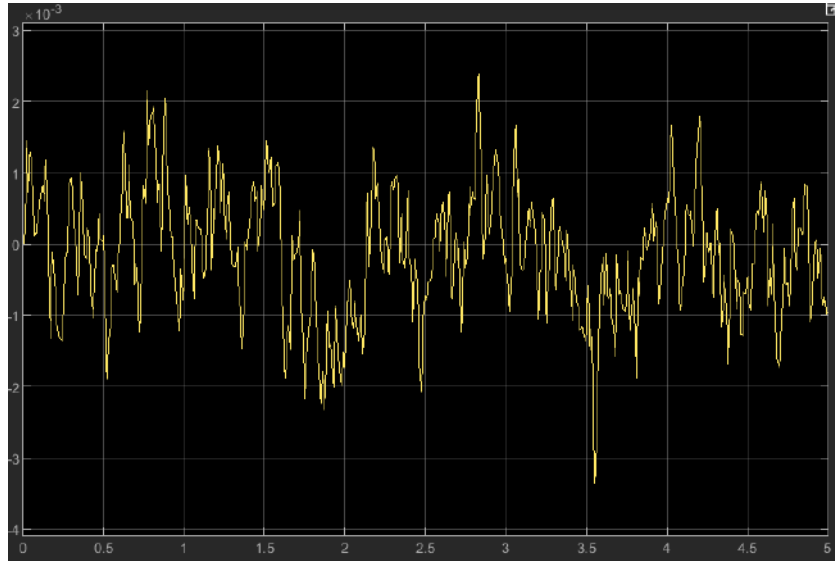


Figure 4.18: Noise

And for example, in the case of NLMPC it will be added only to the controller and not as input to the model of the system, because of course it's only sensor noise and uncertainty, not the real z_r , you can see how it's done here:

```
% z_r at the current instant
md = z_r_disc(k);
if disturbanceFlag == 1
    md = md + noise_disc(k);
end
% computing optimal input
mv = nlmpcmove(nlobj, x, lastmv, yref, md);
u_full = [mv; md];

if disturbanceFlag == 1
    md = md - noise_disc(k);
end
% simulating the system
dyn = @(t,x) quarterCarStates(x, [mv; md]);
[~, x_temp] = ode45(dyn, [0 ts], x);
```

Figure 4.19: Updated code for noise

4.2.1 Road Bump

Starting from LQR:

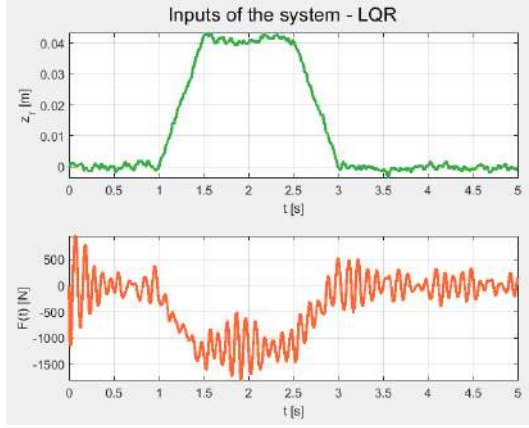


Figure 4.20: Inputs

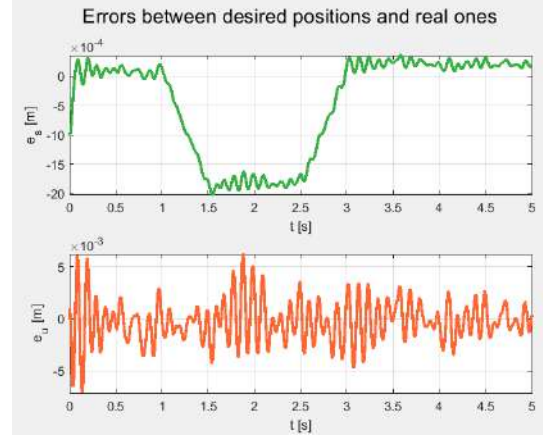


Figure 4.21: Errors

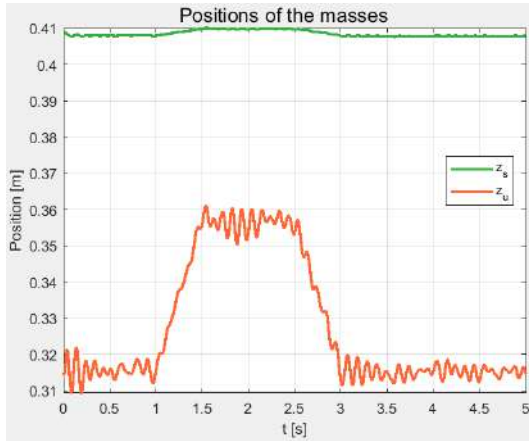


Figure 4.22: Positions

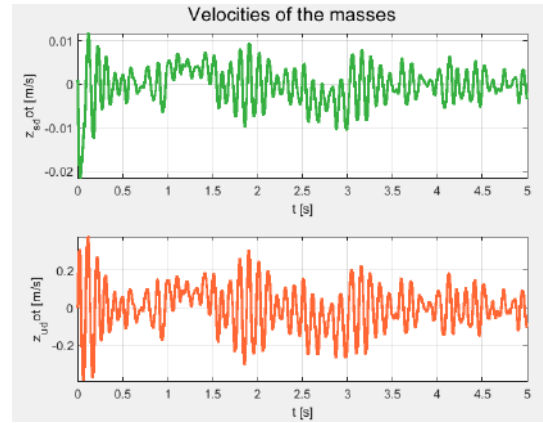


Figure 4.23: Velocities

Considering that in this case we are not mitigating or counteracting the noise in anyway we can notice some differences and of course degradation of performance, but still the controller does its job.

The problem is that since the controller sees the noise then the force $F(t)$ will have a larger variance, thus bigger oscillations. The performances regarding the upper mass are almost the same as without noise in terms of position but the velocity suffers it, thus comfort. The rapid change of $F(t)$ will also trigger the tire:

Table 4.5: Performance indices for active suspension on road bump with disturbance LQR

Metric	Value
RMS_{z_s} [m]	0.001
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.24
$\text{RMS}_{u,r}$ [m]	0.32

With NLMPC we can observe similar problems, thus its clear that the active element will suffer the noise, and may be subject to the shortening of its life also in this case, maybe for its rapid triggering:

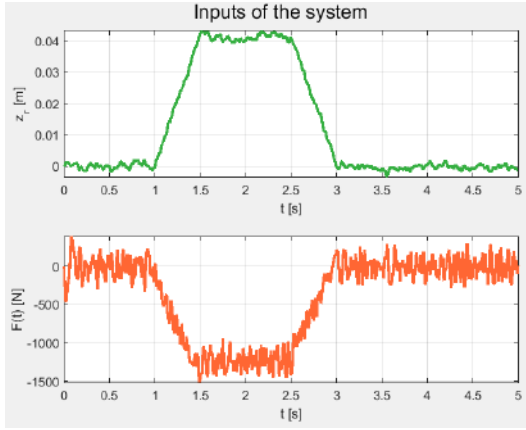


Figure 4.24: Inputs



Figure 4.25: Errors

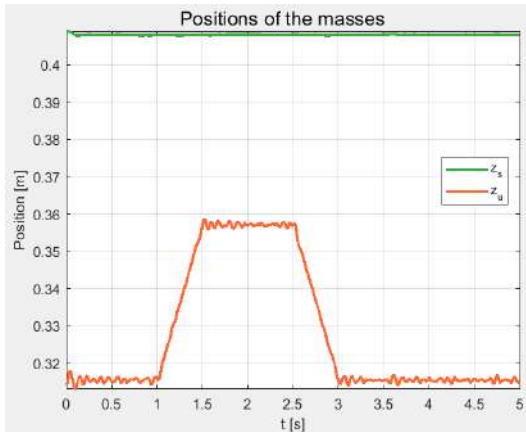


Figure 4.26: Positions

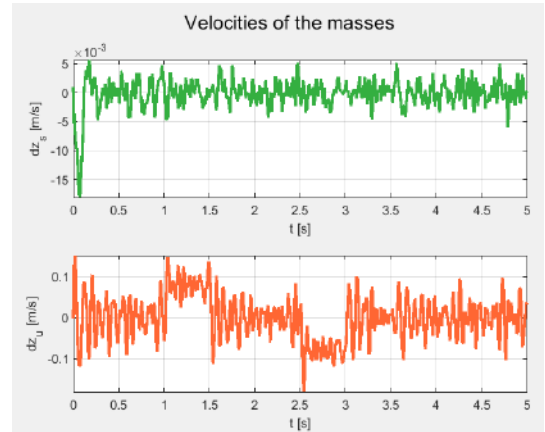


Figure 4.27: Velocities

Table 4.6: Performance indices for active suspension on road bump with disturbance NL MPC

Metric	Value
$\text{RMS}_{z_s} \text{ [m]}$	0.00012
$\text{RMS}_{\ddot{z}_s} \text{ [m/s}^2\text{]}$	0.14
$\text{RMS}_{u,r} \text{ [m]}$	0.32

4.2.2 Off-Road

This case is funny instead, because we can notice that noise's effects are worse in a scenario where the road is simpler with respect to the case where the road is already largely irregular. However of course the bump case is over-complicated, because for example if the road is almost all the time flat then hardly the sensor will have a high uncertainty on the measurement. However:

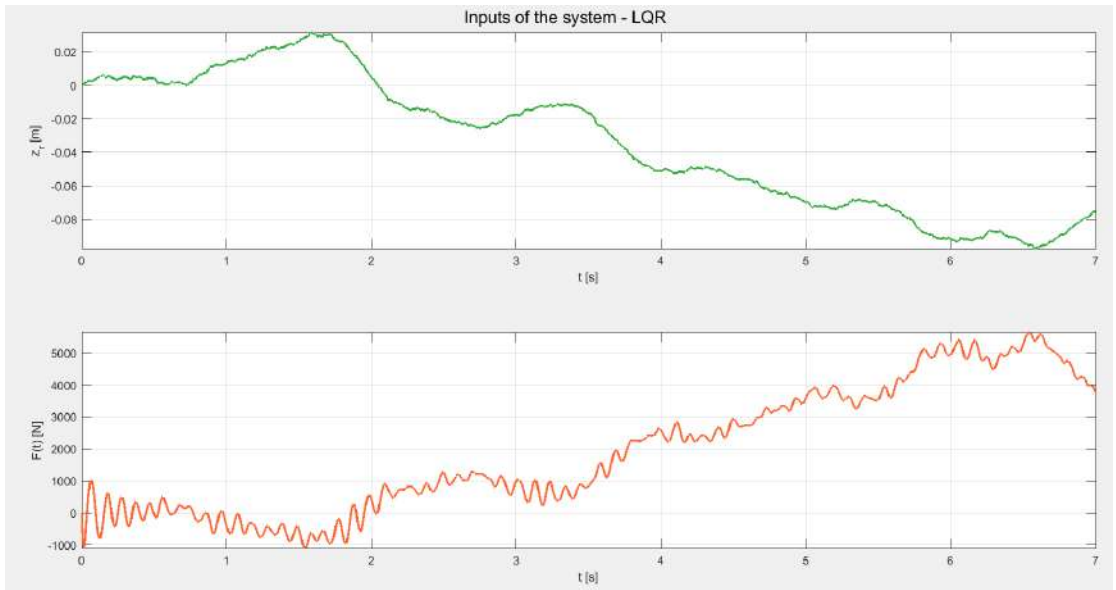


Figure 4.28: Inputs

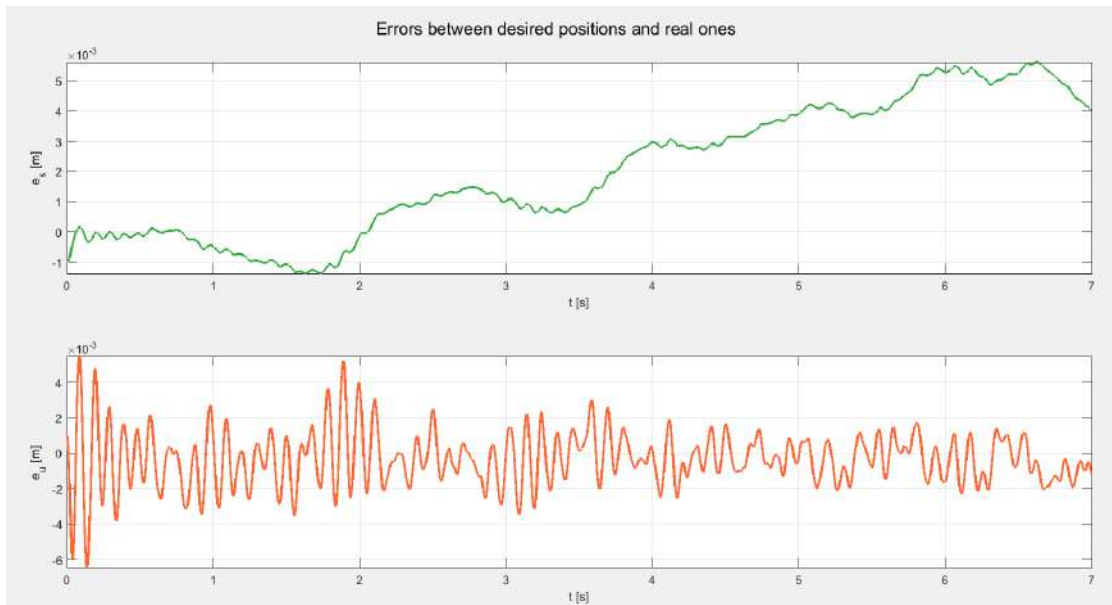


Figure 4.29: Errors

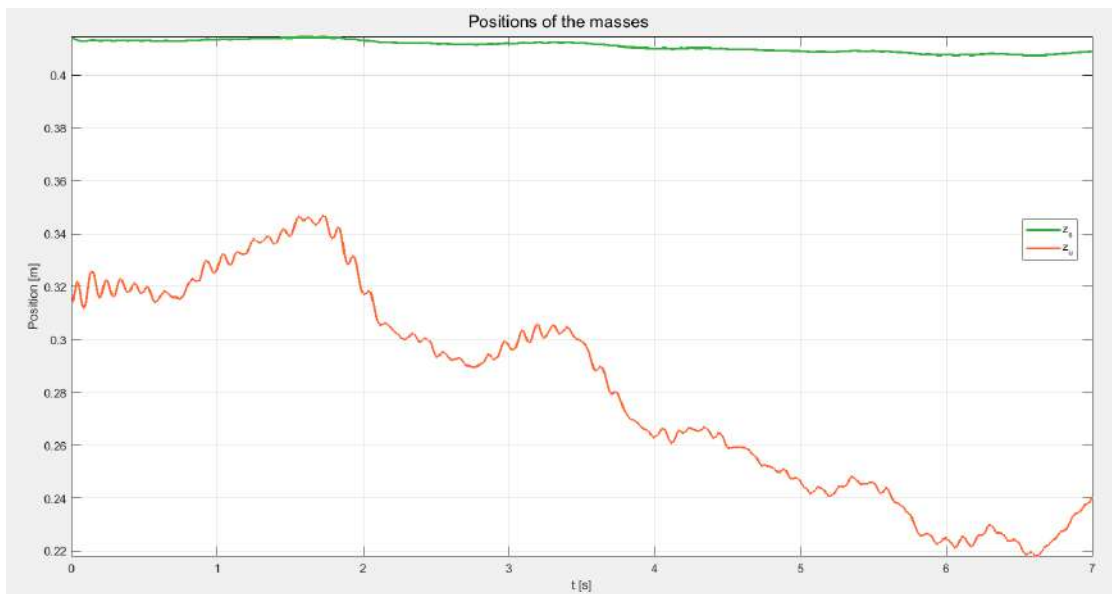


Figure 4.30: Positions

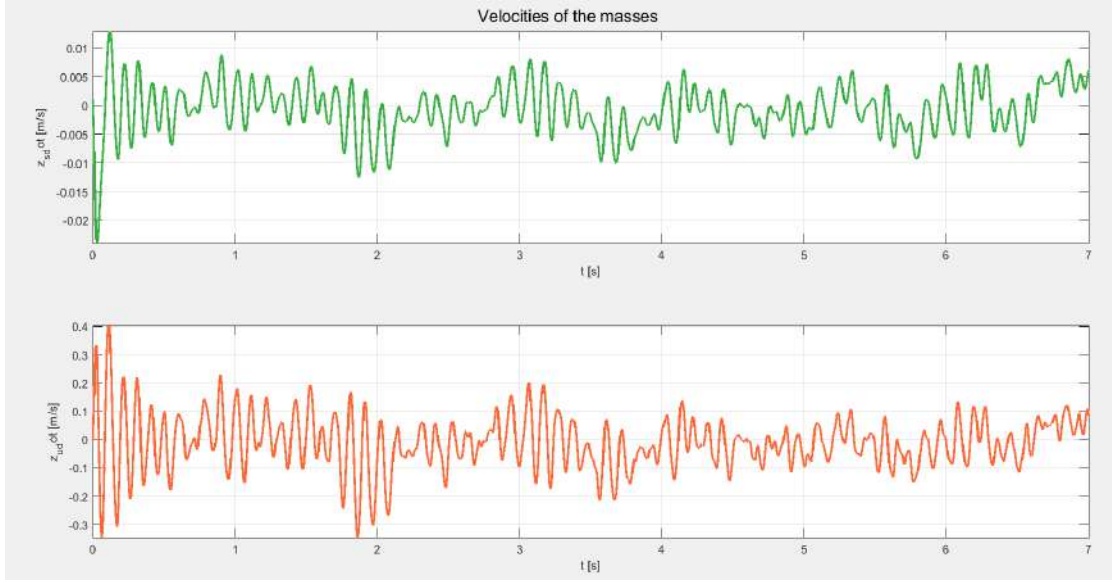


Figure 4.31: Velocities

Table 4.7: Performance indices for active suspension on off-road with disturbance LQR

Metric	Value
RMS_{z_s} [m]	0.0028
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.21
$\text{RMS}_{u,r}$ [m]	0.32

We can observe the same things but slightly better with NLMP:

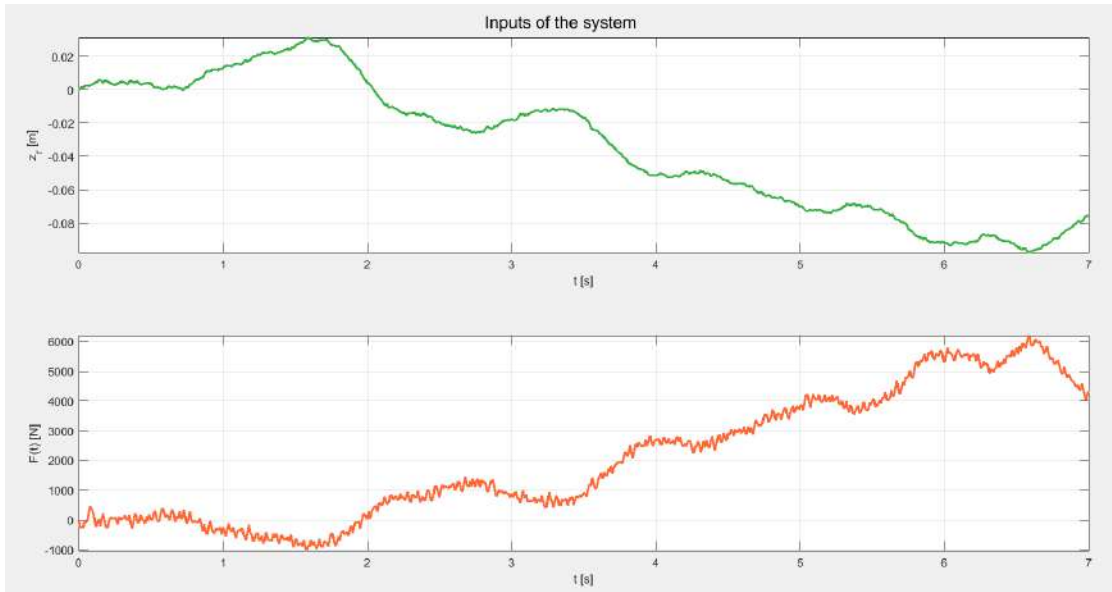


Figure 4.32: Inputs

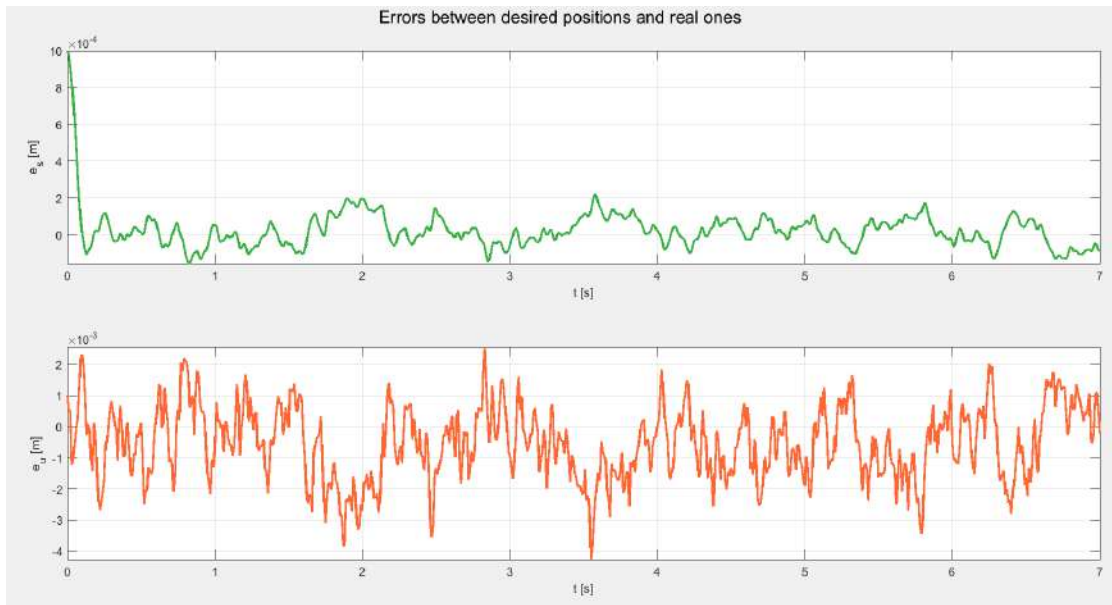


Figure 4.33: Errors

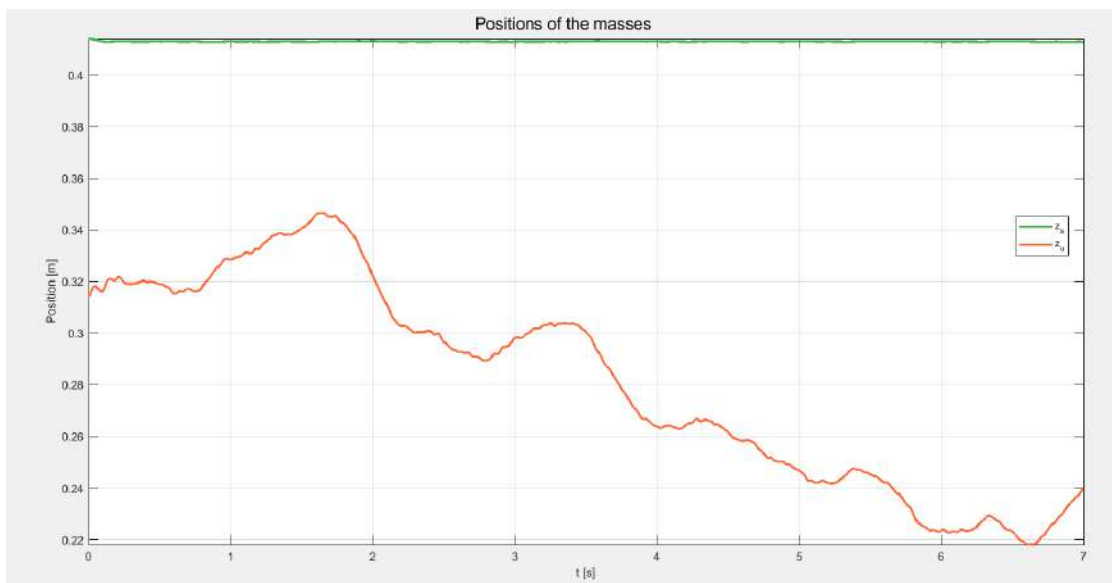


Figure 4.34: Positions



Figure 4.35: Velocities

Table 4.8: Performance indices for active suspension on off-road with disturbance NLMPc

Metric	Value
RMS_{z_s} [m]	0.00011
$\text{RMS}_{\ddot{z}_s}$ [m/s ²]	0.15
$\text{RMS}_{u,r}$ [m]	0.32

4.2.3 Extra, Saturation

In this chapter we've seen different problems of the active system. The fact that it would be better to know, at least approximately the real masses, or the fact that without mitigating noise the piston becomes even more triggered.

But another relevant aspect which has not emerged was *saturation*, so let's see instead, "what happens if for example the actuator is somehow broken and has a lower force limit?" let's consider what happens on the LQR with parametric uncertainties assuming that there's a saturation of $6kN$:

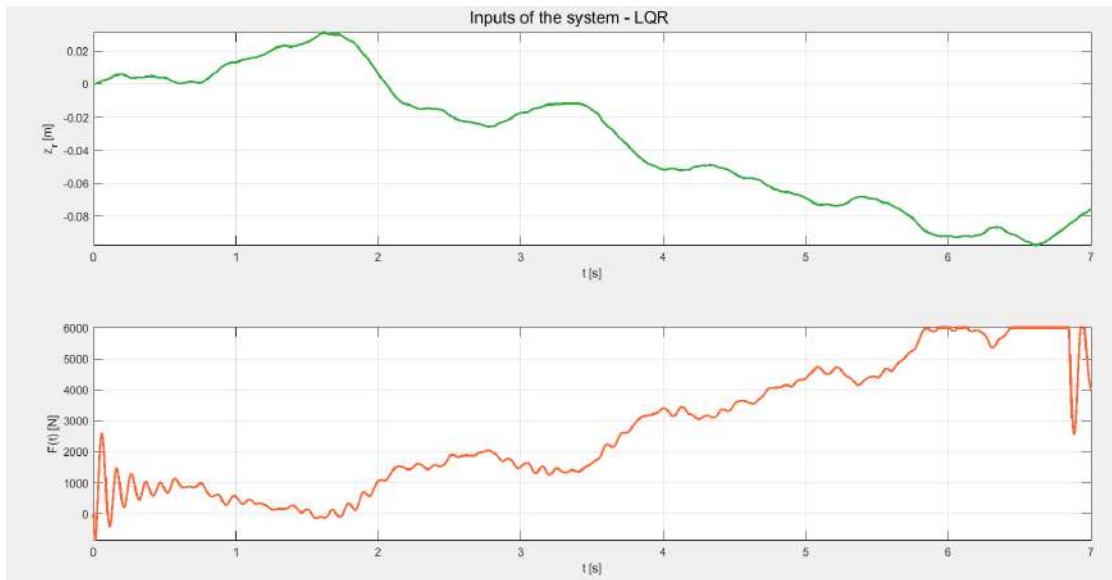


Figure 4.36: Inputs

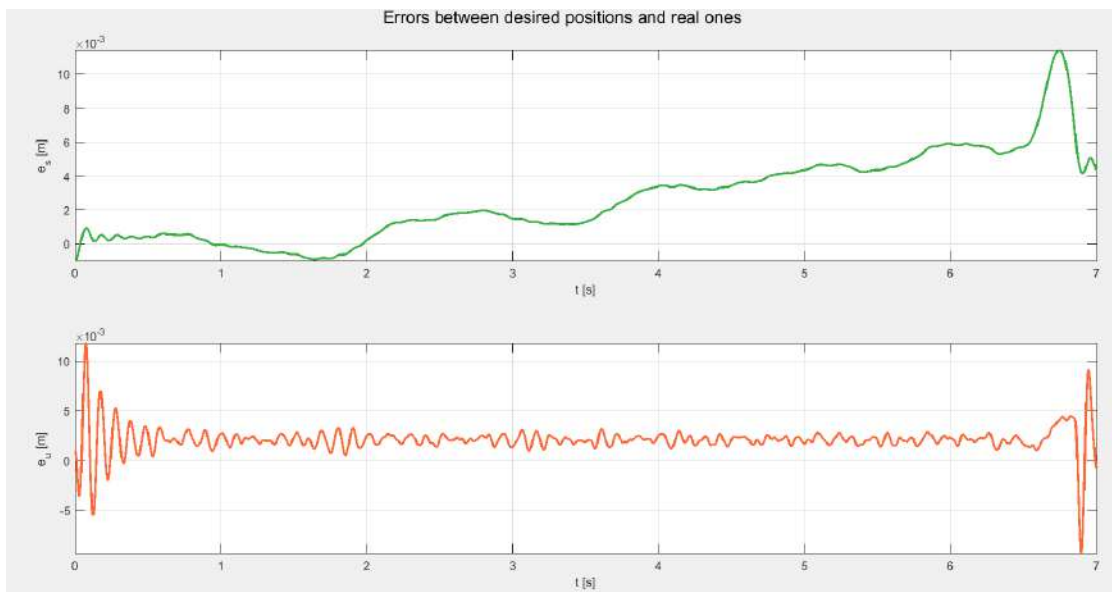


Figure 4.37: Errors

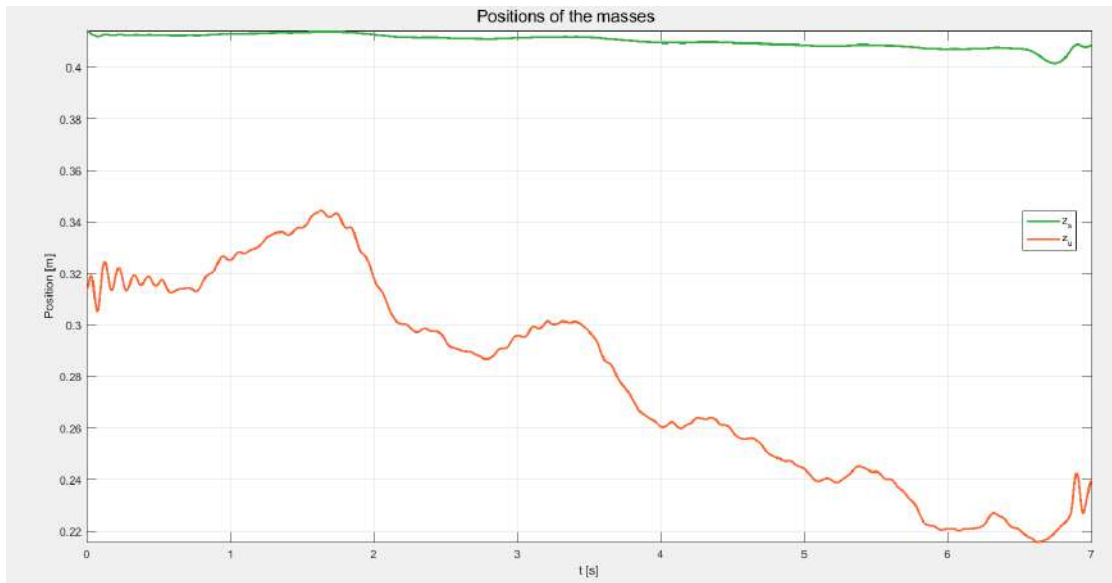


Figure 4.38: Positions

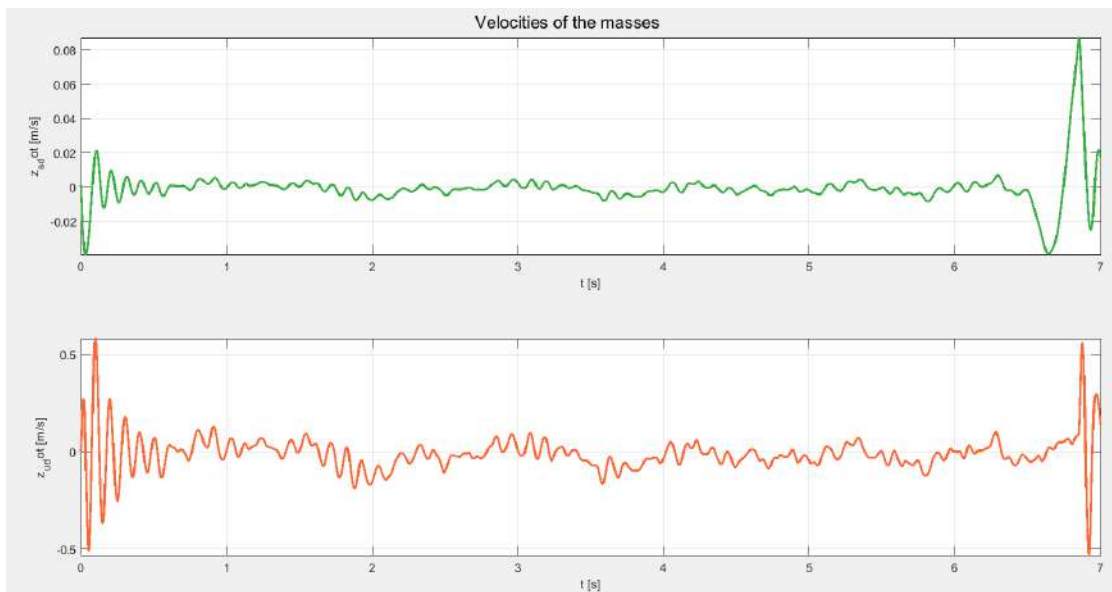


Figure 4.39: Velocities

The effects of saturation are clear, when we do not have enough force to keep counteracting the "disturbance" we can't lift the vehicle and it simply falls down if the road conditions are getting worse in that instant of time. This will cause of course worse performance in terms of comfort, errors, and so on.

If we remember the simulations of LQR and in particular picture 3.11 we can observe that without uncertainties the controller doesn't saturate, but for sure there will be at least one uncertainty over the driver's suspension, thus it's significant to consider them.

Another relevant thing that needs to be highlighted is that saturation not only can occur when there's a failure of the piston. It may happen when the road conditions are so bad that even a perfect actuator cannot compensate for a large excursion of z_r thus this system may work fine in certain scenarios, but of course not everywhere.

Chapter 5

Conclusion

Finally the last chapter, now we'll need to compare the performances of the control schemes by means of the indexes. One thing that can be noticed is that the parameter regarding the tire's displacement is not really changing, being almost fixed to $RMS_{u,r} = 0.32m$ and that's because the suspension is constructed in a way such that the wheel is "passive" and tends to follow the road. Thus we will focus on comfort.

5.1 Passive vs Active

It's the first comparison of course and we want to see how much better are the performances respectively with LQR and NLMPC, thus we'll use this formula as reference to express the result in percentage:

$$\text{Improvement (\%)} = \frac{\text{RMS}_{\text{passive}} - \text{RMS}_{\text{active}}}{\text{RMS}_{\text{passive}}} \times 100 \quad (5.1)$$

Thus, starting with LQR and the two cases, and considering so the tables 2.1, 2.2, 3.1, 3.2 we have:

Table 5.1: Comparison of RMS_{z_s} between Passive and LQR Suspension

Road Profile	Passive	LQR	Improvement (%)
Road Bump	0.024	0.001	95.8
Off-Road	0.05	0.003	94

Table 5.2: Comparison of $\text{RMS}_{\ddot{z}_s}$ between Passive and LQR Suspension

Road Profile	Passive	LQR	Improvement (%)
Road Bump	0.34	0.18	47
Off-Road	0.44	0.15	66

And then NLMPC:

Table 5.3: Comparison of RMS_{z_s} between Passive and NLMPC Suspension

Road Profile	Passive	NLMPC	Improvement (%)
Road Bump	0.024	0.0001	99
Off-Road	0.05	0.0001	99

Table 5.4: Comparison of $\text{RMS}_{\ddot{z}_s}$ between Passive and NLMPC Suspension

Road Profile	Passive	NLMPC	Improvement (%)
Road Bump	0.34	0.05	85
Off-Road	0.44	0.06	86

In this way we are not just comparing numbers on the tables but can say how much of improvement there's thanks to the controllers. LQR is not bad of course but as we have seen, NLMPC offers superior performance, specially in terms of perceived accelerations, which is maybe the more relevant comfort index.

5.2 Degradation of Performance

Now instead we'll quantify the degradation of the indexes from the nominal case to the one in which uncertainties or disturbances are acting. In this case we'll compare how the two controllers behave respectively in nominal conditions and then the particular ones, and we'll consider the effects of one index per time. Since in this case generally there will be a degradation of performances, we'll measure it in this way:

$$\text{Degradation (\%)} = \frac{\text{RMS}_{\text{scenario}} - \text{RMS}_{\text{nominal}}}{\text{RMS}_{\text{nominal}}} \times 100 \quad (5.2)$$

Table 5.5: Comparison of RMS_{z_s} under nominal, parametric uncertainty, and disturbance scenarios

Road Profile	Controller	Nominal [m]	Parametric Uncertainty [m] (%)	Disturbance [m] (%)
Bump	LQR	0.001	0.001 (0%)	0.001 (0%)
Bump	NLMPC	0.0001	0.0001 (0%)	0.001 (0%)
Off-Road	LQR	0.003	0.003 (0%)	0.003 (0%)
Off-Road	NLMPC	0.0001	0.0001 (0%)	0.0001 (0%)

As we can notice, in terms of position performance, neither the uncertainties nor the disturbance affects the results in a relevant way, the controllers can compensate by a bigger force, or a one that changes more rapidly. Instead, in terms of accelerations:

Table 5.6: Comparison of $RMS\ddot{z}_s$ under nominal, parametric uncertainty, and disturbance scenarios

Road Profile	Controller	Nominal	Parametric Uncertainty	Disturbance
		[m]	[m] (%)	[m] (%)
Bump	LQR	0.18	0.27 (50%)	0.24 (33%)
Bump	NLMPC	0.05	0.06 (20%)	0.1 (100%)
Off-Road	LQR	0.15	0.23 (53%)	0.2 (33%)
Off-Road	NLMPC	0.06	0.06 (0%)	0.1 (66%)

Generally, we can state that those result are telling us that perceived comfort, in terms of the vertical acceleration are worse, and like we've said it becomes more evident in case of the simplest scenario, because on an almost flat road the sensor's noise compromises more the performance, with respect to the case in which the road is already highly variable. The parameter uncertainty isn't the real problem of the performance, as we can notice it is the sensor's noise, that affects negatively the comfort.

Of course we have to say that nothing was done to counteract the noise for example and certainly there are techniques that we can use in case of an implementation.

5.3 Final Discussion

For sure this system has his own lacks, we can say that for a road that is highly irregular and maybe also with bigger amplitude, then the controller will not be capable of compensate for it, but maybe it could react with his best even in worse situations.

However, being it a city oriented solution for comfort you would not imagine it (neither the car) on an extreme off-road profile, and maybe it's not even its purpose. For this exists *rally* for example or the *Dakar*, where also the wheel excursion matters and is limited, in this situations comfort is for sure left behind with respect to performance.

Hence, there will be a lot of things to discuss, but I think that having this system during a drive around your city would certainly let you forgot about those boring bumps.

Bibliography

- [1] P. Bolzern, R. Scattolini, and N. Schiavoni, *Fondamenti di controlli automatici*. Milano, Italia: McGraw-Hill Education, 4 ed., 2015.
- [2] H. K. Khalil, *Nonlinear Systems*. Upper Saddle River, NJ, USA: Prentice Hall, 2002.
- [3] G. D. Shelke, A. C. Mitra, and V. R. Varude, “Validation of simulation and analytical model of nonlinear passive vehicle suspension system for quarter car,” *Materials Today: Proceedings*, vol. 5, no. 9, pp. 19294–19302, 2018.
- [4] M. Trzesniowski, *Suspension System*. Wiesbaden, Germany: Springer Vieweg, 2023.
- [5] A. Gargiulo, “Nonlinear dynamics and control of a duffing-type mass-spring-damper system.” Project of Nonlinear Dynamics and Control, 2025.
- [6] D. Rodriguez-Guevara, A. Favela-Contreras, F. Beltran-Carbajal, C. Sotelo, and D. Sotelo, “A differential flatness-based model predictive control strategy for a nonlinear quarter-car active suspension system,” *Mathematics*, vol. 11, no. 4, p. 1067, 2023.